



sec4her: appunti Linux

1. Introduzione a Linux

Cos'è Linux

Linux è un sistema operativo open source basato sul kernel Linux.

Qui è importante fare una distinzione: nel linguaggio comune si dice spesso "Linux" per indicare un intero sistema operativo desktop/server, ma tecnicamente Linux è il kernel, cioè il componente centrale che permette al software di comunicare con l'hardware.

Il kernel si occupa, tra le altre cose, di:

- gestione della CPU;
- gestione della memoria RAM;
- gestione dei processi;
- gestione dei dispositivi hardware;
- gestione dei filesystem;
- gestione della rete;
- gestione dei permessi e della sicurezza;
- comunicazione tra programmi e hardware.

Possiamo immaginare il sistema in questo modo:

```
Applicazioni
↓
Programmi e librerie
↓
Shell / interfacce
↓
Kernel Linux
↓
Hardware
```

Per esempio, quando esegui:

```
ls
```

tu non stai comunicando direttamente con il disco. Il programma `ls` chiede al sistema operativo informazioni sul file system e il kernel si occupa di interagire con il dispositivo.

Kernel

Il kernel è il nucleo centrale di un sistema operativo che gestisce le risorse hardware del computer e permette ai programmi di comunicare con esse.

- Collega hardware e software: riceve le richieste dai programmi e le traduce in comandi per il processore, la memoria e le periferiche.
 - Gestisce la memoria: decide quanta memoria RAM assegnare a ogni programma aperto.
 - Controlla la CPU: regola il tempo di calcolo del processore per fare girare più app insieme.
 - Gestisce i file e i driver: fa funzionare dischi fissi, chiavette USB, stampanti e schede di rete.
-

Distribuzioni Linux

Una distribuzione Linux, o semplicemente distro, è una versione completa di un sistema GNU/Linux.

Esistono moltissime distribuzioni perché il software è open source e può essere modificato, distribuito e combinato in modi diversi.

Le distribuzioni possono differire per:

- gestione dei pacchetti;
- configurazione predefinita;
- ambiente grafico;
- filosofia;
- aggiornamenti;
- strumenti amministrativi;
- supporto hardware;
- stabilità e frequenza degli aggiornamenti.

Distribuzione	A cosa serve / per chi è adatta
Ubuntu	Uso quotidiano, studio, programmazione e desktop. È molto adatta ai principianti.
Linux Mint	Desktop e uso quotidiano. È semplice e familiare per chi arriva da Windows.
Debian	Server e desktop. È famosa per stabilità e affidabilità.
Fedora	Sviluppo, programmazione e utilizzo desktop con software abbastanza aggiornato.
Arch Linux	Utenti esperti che vogliono configurare il sistema in modo molto personalizzato.

Distribuzione	A cosa serve / per chi è adatta
Kali Linux	Cybersecurity, penetration testing e analisi di sicurezza. Non è pensata principalmente per il normale uso quotidiano.

Terminale e shell

Questi due termini vengono spesso confusi.

Terminale

Il **terminale** è un'interfaccia attraverso la quale puoi interagire con il sistema usando testo.

Può essere un'applicazione grafica come Terminal, Konsole, GNOME Terminal...

Il terminale visualizza ciò che la shell produce e permette di inserire comandi.

Shell

La **shell** è il programma che interpreta i comandi che scrivi.

Un esempio molto comune è Bash (Bourne Again SHell). Altre shell sono: zsh, fish e dash.

`echo $0` — Shell utilizzata

Il comando:

```
echo $0
```

può mostrare il nome con cui è stata avviata la shell corrente.

Attenzione: `$0` non è il metodo più affidabile per determinare quale shell sia effettivamente in uso in ogni situazione. Per informazioni sulla shell dell'utente configurata nel sistema si può anche controllare `/etc/passwd` oppure utilizzare `$SHELL`.

Ad esempio:

```
echo $SHELL
```

può restituire:

```
/bin/bash
```

Il rapporto può essere rappresentato così:

```
TU
↓
Terminale
↓
Shell (es. Bash)
```

```
↓  
Programma/comando  
↓  
Kernel  
↓  
Hardware
```

Quindi:

| Il terminale è l'interfaccia. La shell interpreta i comandi.

Command Line Interface (CLI)

È un modo di utilizzare il computer scrivendo comandi testuali invece di utilizzare principalmente finestre, pulsanti e menu.

Per esempio:

```
mkdir documenti
```

chiede al sistema di creare una directory chiamata `documenti`.

Oppure:

```
cd documenti
```

cambia la directory corrente.

La CLI può sembrare più difficile all'inizio rispetto all'interfaccia grafica, ma è estremamente potente.

Con la CLI puoi:

- automatizzare operazioni;
- lavorare velocemente;
- amministrare server;
- manipolare moltissimi file;
- concatenare programmi;
- creare script;
- lavorare su computer senza interfaccia grafica.

Sintassi generale dei comandi

Una struttura molto comune della command line è:

```
comando [opzioni] [argomenti]
```

Per esempio:

```
ls -l /home/utente1
```

Possiamo scomporlo:

```
ls
|
└─ comando: indica il programma o l'azione che vuoi eseguire.

-l
|
└─ opzione: modifica il comportamento di un comando (naturalmente non tutti i comandi richiedono opzioni o argomenti.)
Molti programmi supportano opzioni brevi con un singolo trattino (es. -l) e opzioni lunghe con due trattini (es. --version)

/home/utente1
|
└─ argomento: indica generalmente su cosa deve operare il comando
```

Case sensitivity

Linux è normalmente case-sensitive.

Significa che:

```
file.txt
File.txt
FILE.txt
```

possono essere nomi completamente diversi.

Lo stesso vale per i comandi.

Tab-completion

La tab-completion è una funzionalità della shell che permette di completare automaticamente nomi di:

- comandi;
- file;
- directory;
- percorsi;
- opzioni, in alcuni casi.

Supponiamo di avere:

```
documenti
documenti_importanti
download
```

Se scrivi:

```
cd doc
```

e premi **TAB**, la shell può completare il nome.

Per esempio:

```
cd doc<TAB>
```

potrebbe diventare:

```
cd documenti
```

Se esistono più possibilità, premendo **TAB** puoi visualizzare o attraversare le alternative, a seconda della shell e della configurazione.

È una funzionalità molto importante perché:

1. riduce gli errori di digitazione;
2. fa risparmiare tempo;
3. evita di dover scrivere nomi lunghi;
4. permette di scoprire i nomi disponibili.

| Abituati a usare TAB continuamente.

clear — Pulire il terminale

Il comando:

```
clear
```

pulisce ciò che viene visualizzato nel terminale.

| **Nota:** **clear** non cancella la cronologia dei comandi e non elimina i dati dal sistema. Semplicemente sposta il contenuto precedente fuori dalla schermata.

history — Cronologia dei comandi

La cronologia dei comandi permette di visualizzare i comandi che sono stati eseguiti precedentemente nella shell.

Il comando:

```
history
```

mostra la lista dei comandi eseguiti, numerandoli.

Per rieseguire un comando della cronologia possiamo usare:

```
!3
```

che esegue nuovamente il comando numero `3`.

which — Individuare un comando

Il comando:

```
which nome_comando
```

mostra il percorso dell'eseguibile che la shell troverebbe nel `PATH`.

Esempio:

```
which ls
```

Output possibile:

```
/usr/bin/ls
```

È possibile utilizzare anche:

```
which python3
```

per individuare il percorso dell'eseguibile di Python.

`which` cerca principalmente gli eseguibili attraverso il `PATH`. Per una verifica più generale del comando, è spesso preferibile utilizzare `command -v`.

which -a — Visualizzare più percorsi

Con:

```
which -a ls
```

vengono mostrati tutti gli eseguibili chiamati `ls` che `which` riesce a trovare nelle directory del `PATH`.

Questo può essere utile quando esistono più versioni dello stesso programma.

uptime — Tempo di attività del sistema

Il comando:

```
uptime
```

mostra da quanto tempo il sistema è in esecuzione, oltre ad altre informazioni come il numero di utenti connessi e il carico medio (*load average*).

Esempio:

```
22:30:10 up 3 days, 4:12, 2 users, load average: 0.10, 0.08, 0.05
```

Per visualizzare da quando è stato avviato il sistema:

```
uptime -s
```

Per una forma più leggibile del tempo di attività:

```
uptime -p
```

Comandi di aiuto e documentazione

help

Il comando:

```
help
```

permette di visualizzare informazioni sui comandi interni della shell, cioè comandi che fanno parte direttamente di Bash.

Esempio:

```
help cd
```

mostra le informazioni e le opzioni disponibili per il comando **cd**.

Possiamo anche usare:

```
help
```

senza specificare un comando per visualizzare l'elenco dei comandi per cui è disponibile l'help.

man

Il comando:

```
man
```

significa manual e permette di consultare il manuale di un comando.

Esempio:

```
man ls
```

mostra il manuale del comando `ls`, con la descrizione del comando, le opzioni disponibili e il suo funzionamento.

Ad esempio:

```
man cp
```

mostra tutte le informazioni relative al comando `cp`.

Per uscire dal manuale si preme:

```
q
```

2. Il filesystem di Linux

Struttura del filesystem

Una delle caratteristiche fondamentali di Linux è il filesystem gerarchico.

A differenza di sistemi in cui i dischi possono essere presentati come unità separate, in Linux tutte le risorse del filesystem vengono organizzate all'interno di un unico albero di directory.

Alla base di questo albero troviamo una directory speciale:

```
/
```

Questa directory si chiama root directory.

Attenzione: la root directory `/` non deve essere confusa con l'utente root.

`/` è una directory del filesystem, mentre `root` è il nome dell'utente amministratore.

Root directory `/`

La directory `/` rappresenta il punto più alto della gerarchia del filesystem.

Tutte le altre directory e tutti i file sono raggiungibili, direttamente o indirettamente, partendo da `/`.

Una struttura semplificata può essere rappresentata così:

```
/
├── bin
├── boot
├── dev
├── etc
├── home
│   ├── utente1
│   └── utente2
├── lib
├── tmp
├── usr
└── var
```

Possiamo immaginare il filesystem come un albero:

- `/` è la radice dell'albero;
- le directory sono i rami;
- i file si trovano nelle directory;
- ogni percorso indica come raggiungere una determinata risorsa partendo dalla radice.

Per esempio, consideriamo il file:

```
/home/utente1/documenti/tesi.pdf
```

Il percorso può essere rappresentato in questo modo:

```
/
├── home
│   ├── utente1
│   │   ├── documenti
│   │   └── tesi.pdf
```

Il file `tesi.pdf` si trova quindi:

1. nella directory `/home`;
2. all'interno della directory `utente1`;
3. nella directory `documenti`;
4. ed è infine il file `tesi.pdf`.

Percorsi assoluti e relativi

Per muoversi nel filesystem è importante distinguere tra percorso assoluto e percorso relativo.

Percorso assoluto

Un percorso assoluto parte sempre dalla root `/`.

Per esempio:

```
/home/utente1/documenti/tesi.pdf
```

è un percorso assoluto.

Indica esattamente dove si trova il file, indipendentemente dalla directory in cui ci troviamo attualmente.

Percorso relativo

Un percorso relativo, invece, non parte da `/`.

Viene interpretato a partire dalla directory corrente.

Supponiamo di trovarci in:

```
/home/utente1
```

e di avere:

```
/home/utente1/documenti/tesi.pdf
```

Possiamo raggiungere il file utilizzando:

```
documenti/tesi.pdf
```

Questo è un percorso relativo perché viene interpretato rispetto alla directory corrente.

Directory corrente

Quando utilizziamo la shell, ci troviamo sempre all'interno di una determinata directory.

Questa viene chiamata **directory corrente** o **working directory**.

Per sapere in quale directory ci troviamo possiamo utilizzare:

```
pwd
```

`pwd` significa print working directory

Per esempio:

```
$ pwd  
/home/utente1
```

Questo significa che la directory corrente è:

```
/home/utente1
```

Home directory

Ogni utente normalmente dispone di una **home directory**, cioè una directory personale nella quale vengono conservati i propri file e molte delle proprie impostazioni.

Per esempio:

```
/home/utente1
```

e:

```
/home/utente2
```

sono le home directory di due utenti diversi.

La home directory è importante perché rappresenta lo spazio personale dell'utente.

La shell mette a disposizione una scorciatoia molto comoda:

```
~
```

Il simbolo  rappresenta la **home directory dell'utente corrente**.

Se l'utente corrente è `utente1`:

```
~
```

corrisponde a:

```
/home/utente1
```

Quindi:

```
cd ~
```

significa: vai nella home directory dell'utente corrente.

Anche il comando:

```
cd
```

senza specificare una directory porta normalmente alla home directory dell'utente.

Per esempio:

```
$ pwd  
/tmp  
  
$ cd  
  
$ pwd  
/home/utente1
```

Directory

All'interno dei percorsi Linux possiamo utilizzare anche il simbolo:

```
.
```

Il punto rappresenta la **directory corrente**.

Supponiamo di trovarci in:

```
/home/utente1
```

Il comando:

```
cd .
```

significa: rimani nella directory corrente.

Per esempio, questi due percorsi indicano la stessa posizione:

```
/home/utente1/documenti
```

e:

```
./documenti
```

se la directory corrente è:

```
/home/utente1
```

Directory `..`

Il simbolo:

```
..
```

rappresenta la **directory padre**, cioè la directory che contiene quella corrente.

Supponiamo di trovarci in:

```
/home/utente1/documenti
```

La directory padre è:

```
/home/utente1
```

Quindi:

```
cd ..
```

significa: torna alla directory padre.

Esempio:

```
$ pwd
/home/utente1/documenti
```

```
$ cd ..
```

```
$ pwd
/home/utente1
```

Le principali directory di Linux

La root `/` contiene diverse directory con funzioni specifiche.

È importante conoscere almeno quelle più comuni.

```
/home
```

Contiene le **home directory degli utenti normali**.

Esempio:

```
/home
├── utente1
├── utente2
└── utente3
```

All'interno della propria home, un utente può normalmente creare e modificare i propri file.

Per esempio:

```
/home/utente1/documenti
/home/utente1/immagini
/home/utente1/download
```

/root

La directory:

```
/root
```

è la **home directory dell'utente root**, cioè dell'amministratore del sistema.

Non va confusa con:

```
/
```

Infatti:

```
/      → root directory
/root  → home directory dell'utente root
```

/etc

Contiene principalmente i **file di configurazione del sistema e dei servizi**.

Per esempio, molte impostazioni relative al sistema e ai servizi vengono memorizzate sotto:

```
/etc
```

Questa directory è quindi particolarmente importante per l'amministrazione di Linux.

/bin

Contiene programmi e comandi fondamentali utilizzabili dagli utenti e necessari al funzionamento di base del sistema.

Su molte distribuzioni moderne, `/bin` è collegata o integrata con `/usr/bin` tramite il meccanismo di **usr merge**.

`/sbin`

Contiene tradizionalmente programmi utilizzati soprattutto per l'**amministrazione del sistema**.

Anche `/sbin`, nelle distribuzioni moderne, può essere collegata a directory sotto `/usr`.

`/usr`

Contiene una grande quantità di **programmi, librerie e dati utilizzati dal sistema**.

È una delle directory più importanti di un sistema Linux.

Al suo interno possiamo trovare, ad esempio:

```
/usr/bin
/usr/lib
/usr/share
/usr/local
```

`/var`

Contiene dati che **cambiano durante il funzionamento del sistema**.

Il nome deriva infatti da "variable".

Tra gli esempi troviamo:

```
/var/log
/var/cache
/var/lib
```

In particolare:

```
/var/log
```

contiene molti dei **file di log** prodotti dal sistema e dai servizi.

`/tmp`

È utilizzata per memorizzare **file temporanei**.

Molti programmi utilizzano `/tmp` quando hanno bisogno di creare temporaneamente dei file durante la propria esecuzione.

I file presenti in `/tmp` non devono quindi essere considerati come dati permanenti dell'utente.

`/boot`

Contiene i file necessari alla fase di **avvio del sistema**.

Tra questi possono essere presenti:

- il kernel Linux;
 - file utilizzati dal bootloader;
 - altri dati necessari durante il processo di avvio.
-

`/dev`

Contiene i cosiddetti **device file**, cioè file speciali che permettono di interagire con dispositivi hardware e altre risorse del sistema.

Per esempio, possiamo trovare dispositivi come:

```
/dev/sda  
/dev/nvme0n1
```

che possono rappresentare dispositivi di archiviazione.

Possiamo trovare anche:

```
/dev/null
```

che è un dispositivo speciale utilizzato per scartare dati.

`/proc`

È un **filesystem virtuale** che espone informazioni relative al kernel e ai processi in esecuzione.

Ad esempio:

```
/proc/cpuinfo
```

fornisce informazioni sul processore, mentre:

```
/proc/meminfo
```

fornisce informazioni sulla memoria.

`/proc` è particolare perché i suoi contenuti non corrispondono semplicemente a normali file memorizzati sul disco: molte informazioni vengono generate dinamicamente dal kernel.

/sys

È un altro filesystem virtuale utilizzato per esporre informazioni relative al **kernel, ai dispositivi hardware e ai driver**.

Viene utilizzato soprattutto dal sistema e dagli strumenti di amministrazione per interagire con il modello dei dispositivi del kernel.

/run

Contiene dati **temporanei relativi all'esecuzione del sistema**, come informazioni utilizzate dai servizi e dai processi in esecuzione.

Il suo contenuto normalmente cambia durante il funzionamento del sistema.

/media

Viene utilizzata comunemente come punto di montaggio per dispositivi rimovibili, ad esempio:

- chiavette USB;
- dischi esterni;
- altri supporti.

Per esempio, un dispositivo USB potrebbe essere montato in una directory come:

```
/media/utente/CHIAVETTA
```

/mnt

È una directory tradizionalmente utilizzata come **punto di montaggio temporaneo** per filesystem e dispositivi.

Ad esempio, un amministratore potrebbe montare manualmente un filesystem in:

```
/mnt/dati
```

/opt

È destinata tradizionalmente a **software aggiuntivo** installato al di fuori della gestione standard della distribuzione.

Alcuni programmi possono quindi essere installati in directory come:

```
/opt/nome_programma
```

Schema riassuntivo

Possiamo riassumere le directory principali in questo modo:

```
/
├─ boot    → file necessari all'avvio
├─ dev     → dispositivi e device file
├─ etc     → configurazione del sistema
├─ home    → home degli utenti
├─ media   → dispositivi rimovibili
├─ mnt     → mount point temporanei
├─ opt     → software aggiuntivo
├─ proc    → informazioni virtuali su processi e kernel
├─ root    → home dell'utente root
├─ run     → dati temporanei dei processi in esecuzione
├─ sys     → informazioni su kernel e dispositivi
├─ tmp     → file temporanei
├─ usr     → programmi, librerie e dati del sistema
└─ var     → dati variabili, log, cache, ecc.
```

3. Gestione di utenti e gruppi

Questi comandi permettono di visualizzare gli utenti presenti nel sistema, controllare i relativi privilegi e gestire utenti e gruppi.

id — Identità dell'utente

Il comando `id` mostra le informazioni relative all'utente con cui stiamo eseguendo il comando.

```
id
```

Esempio:

```
uid=1000(utente1) gid=1000(utente1) groups=1000(utente1),27(sudo)
```

Le informazioni principali sono:

- `uid`: User ID, identificativo numerico dell'utente.
- `gid`: Group ID, identificativo numerico del gruppo principale.
- `groups`: gruppi a cui appartiene l'utente.

Per visualizzare le informazioni di un utente specifico:

```
id nome_utente
```

UID e utenti di sistema

In molte distribuzioni Linux, gli utenti normali iniziano tipicamente da UID `1000`, mentre gli UID inferiori sono generalmente utilizzati per utenti di sistema e servizi. Tuttavia, la numerazione può variare in base alla distribuzione e alla configurazione del sistema.

L'utente `root` è un caso particolare:

```
UID = 0
```

L'UID `0` identifica l'utente con privilegi amministrativi completi.

Cybersecurity: `id` è particolarmente utile durante l'enumerazione dei privilegi, perché permette di capire rapidamente quali gruppi possono concedere all'utente capacità aggiuntive.

`sudo` — SuperUser Do

`sudo` permette a un utente autorizzato di eseguire un singolo comando con i privilegi di un altro utente, normalmente `root`.

Esempio:

```
sudo comando
```

In molti sistemi Linux, gli utenti autorizzati a utilizzare `sudo` appartengono al gruppo `sudo` (ad esempio Debian). Tuttavia, non è il gruppo in sé a determinare necessariamente i privilegi: le autorizzazioni di `sudo` sono definite principalmente nella configurazione di `sudoers`.

La configurazione può essere verificata con:

```
sudo visudo
```

`sudo` non significa necessariamente "diventare root". Per impostazione predefinita, esegue il singolo comando con privilegi elevati.

`sudo su` — Aprire una shell come root

```
sudo su
```

Apri una nuova shell con identità `root`.

Una volta dentro, è possibile verificare l'utente corrente con:

```
whoami
```

che normalmente restituisce:

```
root
```

Per uscire dalla shell di root:

```
exit
```

Alternativa consigliata

Un metodo più diretto per ottenere una shell root è:

```
sudo -i
```

Questo avvia una shell di login dell'utente `root`.

`su` — Cambiare utente

Il comando `su` permette di cambiare identità utente.

```
su nome_utente
```

Se non viene specificato alcun utente:

```
su
```

normalmente tenta di passare all'utente `root`.

Per uscire dall'utente corrente e tornare alla shell precedente:

```
exit
```

`su` vs `sudo`

È importante distinguere i due comandi:

- `sudo comando` → esegue un singolo comando con privilegi elevati.
- `sudo -i` → apre una shell di login come `root`.
- `su nome_utente` → cambia utente e normalmente richiede la password dell'utente di destinazione.
- `sudo su` → utilizza `sudo` per avviare una shell come `root`.

Gestione password

`passwd` — Gestire la password di un utente

Il comando:

```
passwd
```

permette di modificare la password dell'utente corrente.

Per modificare la password di un altro utente, normalmente sono necessari privilegi amministrativi:

```
sudo passwd nome_utente
```

`passwd -e` — Forzare il cambio password al prossimo login

Il comando:

```
sudo passwd -e nome_utente
```

fa scadere immediatamente la password dell'utente.

`-e` significa expire.

Di conseguenza, al prossimo accesso, l'utente sarà normalmente obbligato a impostare una nuova password.

`who` — Utenti attualmente connessi

Il comando:

```
who
```

mostra gli utenti attualmente autenticati nel sistema tramite sessioni, indicando normalmente:

- nome utente;
- terminale utilizzato;
- data e ora di accesso;
- eventualmente informazioni aggiuntive relative alla sessione.

`who -a` — Informazioni estese sulle sessioni

Il comando:

```
who -a
```

mostra informazioni più complete relative alle sessioni e ad altri dati gestiti dal sistema.

Attenzione: `who -a` non significa semplicemente "mostra tutti gli utenti presenti sul computer". Mostra informazioni estese sulle sessioni e sugli accessi registrati dal sistema.

`whoami` — Utente corrente

Il comando:

```
whoami
```

mostra il nome dell'utente con cui stiamo eseguendo la shell corrente.

adduser — Creazione degli utenti

Per creare un nuovo utente:

```
sudo adduser nome_utente
```

A seconda della distribuzione, il comando può chiedere:

- una password;
- nome completo;
- altre informazioni facoltative.

Esempio:

```
sudo adduser utente1
```

adduser --allow-bad-names — Nomi utente non conformi

Durante la creazione di un utente, **adduser** può rifiutare alcuni nomi che non rispettano le convenzioni previste dalla distribuzione.

In determinate situazioni è possibile utilizzare:

```
sudo adduser --allow-bad-names nome_utente
```

L'opzione:

```
--allow-bad-names
```

permette di accettare nomi che normalmente **adduser** considererebbe non conformi alle proprie regole.

Nota: non significa che il nome sia necessariamente "pericoloso". Significa che non rispetta le convenzioni normalmente applicate dallo strumento.

Aggiungere un utente al gruppo **sudo**

```
sudo adduser nome_utente sudo
```

In alternativa, è comune utilizzare:

```
sudo usermod -aG sudo nome_utente
```

L'opzione `-aG` significa:

- `a` → aggiungi senza rimuovere gli altri gruppi supplementari;
- `G` → specifica il gruppo o i gruppi supplementari.

È importante usare `-a` insieme a `-G`: `usermod -G` senza `-a` può sostituire l'elenco dei gruppi supplementari dell'utente.

Dopo aver aggiunto l'utente al gruppo, potrebbe essere necessario effettuare un nuovo login perché la modifica venga riconosciuta dalla sessione.

deluser — Eliminazione degli utenti

Per eliminare un utente:

```
sudo deluser nome_utente
```

Questo elimina l'account, ma non elimina automaticamente la directory home dell'utente.

Per eliminare anche la home:

```
sudo deluser --remove-home nome_utente
```

Attenzione: `--remove-home` elimina i dati contenuti nella directory home dell'utente. È quindi un'operazione da eseguire con cautela.

groups — Gruppi dell'utente

Il comando:

```
groups
```

mostra i gruppi a cui appartiene l'utente corrente.

Per un utente specifico:

```
groups nome_utente
```

Per ottenere informazioni più complete su UID, GID e gruppi:

```
id nome_utente
```

addgroup — Creare un gruppo

```
sudo addgroup nome_gruppo
```

addgroup --allow-bad-names — Nomi di gruppo non conformi

Lo stesso concetto può essere applicato ai gruppi:

```
sudo addgroup --allow-bad-names nome_gruppo
```

L'opzione permette di accettare nomi che normalmente non rispettano le regole applicate da **addgroup**.

delgroup — Eliminare un gruppo

```
sudo delgroup nome_gruppo
```

Prima di eliminare un gruppo è opportuno verificare quali utenti ne fanno parte.

Visualizzare gli utenti appartenenti a un gruppo

Il file:

```
/etc/group
```

contiene informazioni sui gruppi locali del sistema.

Per cercare il gruppo **sudo** :

```
cat /etc/group | grep "sudo"
```

Una forma più efficiente è:

```
grep "^sudo:" /etc/group
```

Esempio di output:

```
sudo:x:27:utente1,utente2
```

In questo caso:

- `sudo` → nome del gruppo;
- `x` → campo relativo alla password del gruppo, normalmente non utilizzato;
- `27` → GID del gruppo;
- `utente1,utente2` → utenti elencati come membri supplementari del gruppo.

Metodo alternativo

Per verificare i gruppi di un singolo utente:

```
groups nome_utente
```

oppure:

```
id nome_utente
```

`usermod -g` — Modificare il gruppo primario

Il comando:

```
sudo usermod -g nome_gruppo nome_utente
```

modifica il **gruppo primario** dell'utente.

Attenzione: `-g` non aggiunge semplicemente un gruppo all'utente. Sostituisce il gruppo primario con quello specificato.

Se invece vogliamo aggiungere un gruppo supplementare senza rimuovere gli altri, utilizziamo:

```
sudo usermod -aG nome_gruppo nome_utente
```

4. File e directory

File di configurazione

Linux utilizza numerosi file di configurazione per gestire utenti, gruppi, servizi e impostazioni del sistema.

Molti dei file di configurazione più importanti si trovano nella directory:

```
/etc
```

`/etc` contiene principalmente file di configurazione del sistema e dei servizi installati.

`/etc/passwd` — Informazioni sugli utenti

Il file:

```
/etc/passwd
```

contiene informazioni sugli account presenti nel sistema.

Può essere letto normalmente da tutti gli utenti, mentre la modifica richiede privilegi amministrativi.

Per visualizzarlo:

```
cat /etc/passwd
```

Una riga tipica è:

```
mario:x:1000:1000:Mario Rossi:/home/mario:/bin/bash
```

I campi sono separati dal carattere `:`:

```
nome_utente : password : UID : GID : nome_completo : home : shell
```

Nel dettaglio:

- `nome_utente` → nome dell'account.
- `x` → indica che le informazioni relative alla password non sono memorizzate direttamente in `/etc/passwd`; normalmente sono presenti in `/etc/shadow`.
- `UID` → identificativo numerico dell'utente.
- `GID` → identificativo del gruppo principale dell'utente.
- `nome_completo` → campo descrittivo dell'utente.
- `home` → percorso assoluto della directory personale dell'utente.
- `shell` → shell assegnata all'utente, ad esempio `/bin/bash`.

Cybersecurity: `/etc/passwd` è uno dei primi file da analizzare durante l'enumerazione locale di un sistema, perché permette di identificare gli account presenti e alcune delle loro caratteristiche.

`/etc/shadow` — Password degli utenti

Il file:

```
/etc/shadow
```

contiene le informazioni relative alle password degli utenti locali, inclusi i hash delle password, oltre ad altre informazioni sulla scadenza e sulla validità delle password.

Per motivi di sicurezza, normalmente può essere letto solamente da `root` o da processi/utenti con privilegi appropriati.

Per esempio, una riga può avere una struttura simile a:

```
mario:$6$...:20000:0:99999:7:::
```

Il secondo campo contiene normalmente l'hash della password.

Nota: `/etc/shadow` non contiene la password in chiaro. Contiene normalmente un valore derivato dalla password tramite una funzione di hashing e i relativi parametri.

Per visualizzarlo con privilegi amministrativi:

```
sudo cat /etc/shadow
```

Cybersecurity: `/etc/shadow` è un file estremamente sensibile. L'accesso al suo contenuto può essere rilevante durante attività di amministrazione, auditing e analisi di sicurezza.

`/etc/gshadow` — Informazioni sui gruppi

Il file:

```
/etc/gshadow
```

contiene informazioni relative ai gruppi locali, comprese eventuali informazioni sulle password dei gruppi e sui relativi membri.

Come `/etc/shadow`, è un file sensibile e normalmente non è accessibile agli utenti normali.

Per visualizzarlo:

```
sudo cat /etc/gshadow
```

```
nome_gruppo : password : amministratori : membri
```

Dove:

- `nome_gruppo` → nome del gruppo.
- `password` → informazioni relative alla password del gruppo.
- `amministratori` → utenti autorizzati a gestire il gruppo.
- `membri` → utenti appartenenti al gruppo.

Cybersecurity: durante l'analisi di un sistema è importante considerare anche i gruppi, perché l'appartenenza a determinati gruppi può concedere privilegi aggiuntivi.

ls — Visualizzare il contenuto di una directory

Il comando:

```
ls
```

mostra il contenuto della directory corrente.

ls directory — Visualizzare il contenuto di una directory specifica

È possibile specificare una directory come argomento:

```
ls nome_directory
```

È possibile utilizzare anche un percorso assoluto:

```
ls /etc
```

In questo caso vengono mostrati i contenuti della directory `/etc` senza doverci entrare.

ls -l — Visualizzazione dettagliata

L'opzione `-l` (*long format*) mostra il contenuto in formato dettagliato.

```
ls -l
```

Esempio:

```
-rw-r--r-- 1 mario mario 1250 Oct  3 20:15 note.txt
drwxr-xr-x 2 mario mario 4096 Oct  3 19:30 documenti
```

Tra le informazioni visualizzate troviamo:

- tipo di file e permessi;
- numero di hard link;
- proprietario;
- gruppo;
- dimensione;
- data e ora dell'ultima modifica;
- nome del file o della directory.

Attenzione: la data mostrata da `ls -l` è normalmente quella dell'ultima modifica del contenuto, non necessariamente quella di creazione del file.

`ls -lt` — Ordinare per ultima modifica

Il comando:

```
ls -lt
```

mostra i file in formato dettagliato e li ordina in base alla data di ultima modifica, dal più recente al meno recente.

Le opzioni sono:

- `l` → visualizzazione dettagliata;
- `t` → ordina per tempo di modifica.

Quindi mostra normalmente prima gli elementi modificati più recentemente.

Nota: `ls` non ordina per "data di creazione" con `-t`. L'ordinamento standard di `-t` utilizza il modification time (mtime).

`ls -ltr` — Ordinare per ultima modifica in ordine inverso

Il comando:

```
ls -ltr
```

è equivalente a:

```
ls -l -t -r
```

Le opzioni sono:

- `l` → visualizzazione dettagliata;
- `t` → ordina per tempo di modifica;
- `r` → inverte l'ordine.

Di conseguenza, gli elementi modificati meno recentemente vengono mostrati prima e quelli modificati più recentemente dopo.

`ls -la` — Mostrare anche i file nascosti

Il comando:

```
ls -la
```

mostra tutti gli elementi della directory, compresi quelli nascosti.

Le opzioni sono:

- `l` → formato dettagliato;
- `a` (*all*) → include anche i file nascosti.

In Linux, un file viene normalmente considerato nascosto quando il suo nome inizia con:

```
.
```

Esempi:

```
.bashrc  
.profile  
.config
```

Con `ls -la` vengono mostrati anche:

```
.  
..
```

dove:

- `.` → directory corrente;
- `..` → directory padre.

`ls -d` — Visualizzare la directory stessa

Normalmente:

```
ls /etc
```

mostra il contenuto di `/etc`.

Con:

```
ls -d /etc
```

viene invece mostrata la directory `/etc` stessa, senza elencarne il contenuto.

Questo può essere particolarmente utile insieme a `-l`:

```
ls -ld /etc
```

che permette di visualizzare i permessi, il proprietario, il gruppo e altre informazioni relative alla directory stessa.

stat — Informazioni dettagliate su file e directory

Il comando:

```
stat nome_file
```

mostra informazioni dettagliate relative a un file.

Esempio:

```
stat note.txt
```

Può mostrare informazioni come:

- dimensione;
- inode;
- permessi;
- proprietario;
- gruppo;
- access time (atime);
- modification time (mtime);
- change time (ctime).

Può essere utilizzato anche sulle directory:

```
stat /etc
```

Nota: `ctime` non significa necessariamente "creation time". In Linux indica il change time, cioè il momento in cui sono cambiati i metadati dell'inode, ad esempio permessi, proprietario o altri attributi. Il supporto a una vera data di creazione (`birth time`) dipende dal filesystem.

tree — Visualizzare la struttura delle directory

Il comando:

```
tree
```

mostra graficamente la struttura di directory e file.

Esempio:

```
tree nome_directory
```

Output:

```
nome_directory
├── file1.txt
├── file2.txt
├── documenti
│   ├── documento1.txt
│   └── documento2.txt
└── immagini
    └── foto.jpg
```

Opzioni utili

Per visualizzare anche i file nascosti:

```
tree -a
```

Per mostrare anche i permessi:

```
tree -p
```

Per limitare la profondità a due livelli:

```
tree -L 2
```

Per mostrare solamente le directory:

```
tree -d
```

Per cercare elementi secondo un pattern:

```
tree -P "f*"
```

- **P** (*pattern*) permette di specificare un pattern per i nomi da visualizzare.

I caratteri jolly come ***** possono quindi essere utilizzati per effettuare una selezione.

tr — Sostituire caratteri

Il comando:

```
tr
```

permette di trasformare o eliminare caratteri dall'input.

`tr` lavora normalmente sull'input standard, quindi è comune utilizzarlo attraverso una pipe:

```
echo "ciao" | tr "abcd" "1234"
```

Output:

```
3i13
```

Il funzionamento è basato sulla corrispondenza tra i caratteri dei due insiemi.

```
a → 1  
b → 2  
c → 3  
d → 4
```

Quindi:

```
ciao
```

diventa:

```
3i13
```

I caratteri non presenti nel primo insieme, come `o`, rimangono invariati.

`tr -t` — Troncare il secondo insieme di caratteri

L'opzione:

```
-t
```

(*truncate-set1*) limita il secondo insieme alla lunghezza del primo.

Esempio:

```
echo "ciaofg" | tr -t "abcdef" "1234"
```

In questo caso il secondo insieme:

```
1234
```

viene utilizzato senza estenderlo oltre la lunghezza necessaria.

`tr -t` non significa semplicemente "sostituisci solo i caratteri trovati": modifica il modo in cui viene gestita la lunghezza dei due insiemi. È quindi importante distinguere questa opzione da `tr` normale.

`tr -d` — Eliminare caratteri

L'opzione:

```
-d
```

permette di eliminare i caratteri specificati.

Esempio:

```
echo "ciao" | tr -d "abcd"
```

Output:

```
io
```

Infatti:

```
c → eliminato  
i → rimane  
a → eliminato  
o → rimane
```

Risultato:

```
io
```

`cat` — Visualizzare il contenuto di un file

Il comando:

```
cat nome_file
```

visualizza il contenuto del file direttamente nel terminale.

Esempio:

```
cat /etc/hostname
```

Per file molto lunghi è spesso più comodo utilizzare:

```
less /etc/passwd
```

cd — Cambiare directory

Il comando:

```
cd
```

significa **Change Directory** e permette di spostarsi all'interno del filesystem.

Per entrare in una directory:

```
cd nome_directory
```

Esempio:

```
cd Documents
```

È possibile utilizzare anche un percorso assoluto:

```
cd /etc
```

cd senza argomenti — Tornare alla home

Se eseguiamo:

```
cd
```

senza specificare una directory, la shell ci porta nella **home directory dell'utente corrente**.

Per esempio, se l'utente è `mario`:

```
/home/mario
```

È equivalente a:

```
cd ~
```

cd . — Directory corrente

Il comando:

```
cd .
```

significa "entra nella directory corrente".

Di fatto **non ci sposta da nessuna parte**, perché `.` rappresenta già la directory corrente.

Per conoscere il percorso della directory corrente si utilizza invece:

```
pwd
```

`cd ..` — Directory padre

Il simbolo:

```
..
```

rappresenta la directory padre, cioè quella che contiene la directory corrente.

Esempio:

Se siamo in:

```
/home/mario/Documents
```

eseguendo:

```
cd ..
```

torniamo a:

```
/home/mario
```

`cd ../..` — Salire di due directory

È possibile utilizzare più volte `..` per risalire nella struttura.

Esempio:

```
cd ../..
```

Se partiamo da:

```
/home/mario/Documents/Progetti
```

dopo `cd ../..` arriviamo a:

```
/home/mario
```

Infatti:

```
Progetti → Documents → mario
```

Gestione di file, directory, alias e collegamenti

Questi comandi permettono di creare, copiare, spostare, rinominare ed eliminare file e directory. Permettono inoltre di creare alias, collegamenti e di controllare lo spazio disponibile e lo stato del sistema.

alias — Creare e visualizzare alias

Il comando:

```
alias
```

senza argomenti mostra gli alias attualmente definiti nella shell.

Un **alias** permette di associare un comando o una sequenza di comandi a un nome più breve.

Esempio:

```
alias x='firefox'
```

Da questo momento, nella sessione corrente, è possibile eseguire:

```
x
```

che equivale a:

```
firefox
```

Questo alias è normalmente temporaneo: viene perso quando termina la sessione della shell, a meno che non venga salvato nella configurazione della shell, ad esempio in `~/.bashrc`.

Per eliminare un alias:

```
unalias x
```

mkdir — Creare una directory

Il comando:

```
mkdir nome_directory
```

crea una nuova directory.

Esempio:

```
mkdir documenti
```

crea:

```
documenti/
```

Se il nome contiene spazi, è necessario racchiuderlo tra virgolette oppure effettuare l'escape dello spazio:

```
mkdir "nome directory"
```

oppure:

```
mkdir nome\ directory
```

mkdir — Creare directory annidate

È possibile specificare un percorso per creare una directory all'interno di un'altra:

```
mkdir padre/figlia
```

Questo comando funziona se la directory **padre** esiste già.

Se vogliamo creare contemporaneamente tutte le directory necessarie, utilizziamo **-p** :

```
mkdir -p dir1/dir2/dir3
```

Verrà creata la struttura:

```
dir1/  
└─ dir2/  
    └─ dir3/
```

L'opzione `-p` permette quindi di creare anche le directory genitore che non esistono.

`mkdir --mode` — Impostare i permessi durante la creazione

È possibile specificare i permessi della nuova directory utilizzando:

```
mkdir --mode=0652 nome_directory
```

oppure:

```
mkdir -m 0652 nome_directory
```

I permessi `0652` corrispondono a:

```
owner  → rw-  
group  → -wx  
others → r--
```

Attenzione: i permessi di una directory hanno un significato leggermente diverso rispetto a quelli di un file. In particolare, `x` su una directory permette di attraversarla/accedere agli elementi conoscendone il nome.

Inoltre, i permessi effettivi possono essere influenzati dalla **umask** del processo.

`rmdir` — Eliminare una directory vuota

Il comando:

```
rmdir nome_directory
```

elimina una directory solo se è vuota.

Esempio:

```
rmdir documenti
```

Se `documenti` contiene file o altre directory, il comando non la eliminerà.

`rm` — Eliminare file

Per eliminare un file:

```
rm nome_file
```

Esempio:

```
rm documento.txt
```

Il file viene rimosso.

`rm` è un comando da utilizzare con attenzione: normalmente un file eliminato con `rm` non viene spostato nel cestino.

`rm -r` — Eliminare directory e contenuto

L'opzione `-r` (*recursive*) permette di eliminare una directory insieme al suo contenuto.

```
rm -r nome_directory
```

Ad esempio:

```
rm -r documenti
```

può eliminare:

```
documenti/  
├─ file1.txt  
├─ file2.txt  
└─ sottocartella/  
    └─ file3.txt
```

Attenzione: `rm -r` può eliminare una grande quantità di dati. L'utilizzo di `sudo rm -r` richiede particolare cautela.

`mv` — Spostare file e directory

Il comando:

```
mv origine destinazione
```

permette di spostare o rinominare file e directory.

Esempio:

```
mv nome_file.txt /home/mario/documenti/
```

sposta `nome_file.txt` nella directory specificata.

Non è obbligatorio utilizzare un percorso assoluto. È possibile utilizzare anche un percorso relativo:

```
mv nome_file.txt documenti/
```

mv — Rinominare un file o una directory

mv viene utilizzato anche per rinominare.

```
mv nome_vecchio.txt nome_nuovo.txt
```

In questo caso il file non viene spostato in un'altra directory: viene semplicemente rinominato.

Lo stesso principio vale per le directory:

```
mv vecchia_directory nuova_directory
```

cp — Copiare un file

Il comando:

```
cp origine destinazione
```

crea una copia del file.

Esempio:

```
cp nome_file.txt /home/mario/documenti/
```

È possibile utilizzare anche percorsi relativi:

```
cp nome_file.txt documenti/
```

Per impostazione predefinita, **cp** copia i dati e normalmente mantiene i permessi di base del file, mentre proprietario e gruppo possono dipendere dall'utente che esegue la copia e dal sistema.

cp -r — Copiare directory

Per copiare una directory e tutto il suo contenuto:

```
cp -r nome_directory destinazione/
```

Esempio:

```
cp -r documenti backup/
```

copierà `documenti` all'interno di `backup`.

`cp -i` — Chiedere conferma prima di sovrascrivere

L'opzione:

```
-i
```

(*interactive*) chiede conferma prima di sovrascrivere un file esistente.

Esempio:

```
cp -i file.txt backup/
```

Se nella destinazione esiste già un file con lo stesso nome, verrà richiesta una conferma.

`cp -u` — Copiare solo se necessario

L'opzione:

```
-u
```

(*update*) fa sì che `cp` copi il file solamente quando quello di origine è più recente rispetto a quello già presente nella destinazione, oppure quando il file di destinazione non esiste.

Esempio:

```
cp -u file.txt backup/
```

È utile, ad esempio, per aggiornare una copia senza sovrascrivere inutilmente file già aggiornati.

`dd` — Copiare e convertire dati a blocchi

Il comando:

```
dd
```

permette di leggere dati da un input e scriverli in un output, lavorando a blocchi.

La sintassi di base è:

```
dd if=input of=output
```

Dove:

- `if` → *input file*, sorgente dei dati;
- `of` → *output file*, destinazione dei dati;
- `bs` → *block size*, dimensione del blocco;
- `count` → numero di blocchi da elaborare.

Esempio:

```
dd if=x.txt of=copiato.txt bs=4 count=2
```

In questo caso:

```
bs=4
```

significa che ogni blocco è di 4 byte.

```
count=2
```

significa che vengono copiati 2 blocchi.

Quindi vengono elaborati:

```
4 × 2 = 8 byte
```

Se `x.txt` contiene 14 byte, `copiato.txt` conterrà solamente i primi 8 byte dell'input.

Nota: il valore di `bs` è espresso in byte, salvo l'utilizzo di suffissi come `K`, `M`, `G` ecc.

`dd conv=ucase` — Convertire i caratteri in maiuscolo

L'opzione:

```
conv=ucase
```

permette di convertire le lettere minuscole in maiuscole durante la copia.

Esempio:

```
dd if=x.txt of=copiato.txt conv=ucase
```

Se `x.txt` contiene:

```
ciao mondo
```

il file di destinazione conterrà:

```
CIAO MONDO
```

`dd skip` — Saltare blocchi dell'input

L'opzione:

```
skip=x
```

indica a `dd` di saltare `x` blocchi all'inizio dell'input prima di iniziare la lettura.

Esempio:

```
dd if=x.txt of=copiato.txt bs=4 skip=2
```

Con:

```
bs=4  
skip=2
```

vengono saltati:

```
2 × 4 = 8 byte
```

all'inizio dell'input.

Attenzione: `skip` indica un numero di blocchi, non direttamente un numero di byte. La quantità di byte dipende quindi da `bs`.

`dd seek` — Saltare blocchi nell'output

L'opzione:

```
seek=x
```

si applica all'output.

Indica a `dd` di saltare `x` blocchi all'inizio dell'output prima di iniziare a scrivere.

Esempio:

```
dd if=x.txt of=copiato.txt bs=4 seek=2
```

Con `bs=4` :

```
seek=2
```

significa che la scrittura inizierà dopo:

```
2 × 4 = 8 byte
```

dall'inizio dell'output.

Nota: `seek` utilizza blocchi, non byte direttamente. Se il file di output esiste già, il comportamento dipende anche dalle opzioni utilizzate da `dd`, come `conv=notrunc`.

`df` — Spazio disponibile nei filesystem

Il comando:

```
df
```

mostra informazioni relative allo spazio utilizzato e disponibile nei filesystem montati.

Non mostra semplicemente "la grandezza delle partizioni del disco": mostra lo spazio disponibile sui filesystem.

Per una visualizzazione più leggibile:

```
df -h
```

L'opzione `-h` (*human-readable*) utilizza unità più facilmente leggibili, come:

```
KiB  
MiB  
GiB
```

Esempio:

```
df -h
```

può mostrare:

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	50G	20G	28G	42%	/

du — Spazio utilizzato da file e directory

Il comando:

```
du
```

mostra quanto spazio su disco viene utilizzato da file e directory.

Per una visualizzazione più leggibile:

```
du -h
```

Per ottenere il totale di una directory:

```
du -sh nome_directory
```

Dove:

- **s** → mostra solamente il totale;
- **h** → formato leggibile.

ln — Creare collegamenti

Il comando:

```
ln
```

permette di creare collegamenti (*link*) a file.

Esistono principalmente due tipi:

- **hard link**;
- **symbolic link**, o symlink.

`ln` — Hard link

La sintassi è:

```
ln file_originale nome_link
```

Esempio:

```
ln file.txt collegamento.txt
```

Un hard link non è semplicemente una "scorciatoia": i due nomi fanno riferimento allo **stesso inode e agli stessi dati**.

Se eliminiamo `file.txt`, i dati rimangono accessibili tramite `collegamento.txt`, finché esiste almeno un hard link che li riferisce.

Gli hard link hanno alcune limitazioni:

- normalmente non possono essere creati per directory;
- devono generalmente trovarsi sullo stesso filesystem.

`ln -s` — Collegamento simbolico

Per creare un collegamento simbolico:

```
ln -s percorso_originale nome_link
```

Esempio:

```
ln -s /var/cache/apt/archives ~/pacchetti
```

In questo caso:

```
~/pacchetti
```

è un collegamento simbolico che punta a:

```
/var/cache/apt/archives
```

Un symlink contiene quindi un riferimento al percorso dell'oggetto di destinazione.

A differenza di un hard link, può normalmente puntare anche a una directory e può attraversare filesystem differenti.

Gestione proprietari

Questi comandi permettono di gestire il proprietario e il gruppo associati a file e directory.

chown — Cambiare proprietario e gruppo

Il comando:

```
chown
```

significa **change owner** e permette di modificare il proprietario di un file o di una directory.

La sintassi più comune è:

```
sudo chown nome_utente:nome_gruppo nome_file
```

Esempio:

```
sudo chown mario:pentest report.txt
```

In questo caso:

- il nuovo proprietario è `mario` ;
- il nuovo gruppo proprietario è `pentest` ;
- il file interessato è `report.txt` .

Quindi sia il proprietario sia il gruppo vengono modificati.

Per cambiare il proprietario di un file normalmente sono necessari privilegi amministrativi, ad esempio `root` o `sudo` .

chown :gruppo — Cambiare solamente il gruppo

È possibile utilizzare `chown` senza specificare un nuovo proprietario:

```
sudo chown :nome_gruppo nome_file
```

Esempio:

```
sudo chown :pentest report.txt
```

In questo caso:

- il proprietario rimane invariato;

- il gruppo proprietario diventa `pentest` .

`chgrp` — Cambiare il gruppo proprietario

Il comando:

```
chgrp
```

significa **change group**.

Permette di modificare il gruppo proprietario di un file o di una directory.

```
sudo chgrp nome_gruppo nome_file
```

È sostanzialmente un modo specifico per modificare il gruppo, mentre `chown` può modificare sia proprietario sia gruppo.

`chown -R` e `chgrp -R` — Operazioni ricorsive

L'opzione:

```
-R
```

(*recursive*) permette di applicare la modifica anche al contenuto di una directory.

Esempio:

```
sudo chown -R mario:pentest progetto/
```

modifica ricorsivamente proprietario e gruppo di `progetto/` e di ciò che contiene.

Attenzione: utilizzare `-R` con `chown` o `chmod` su directory importanti del sistema può causare problemi molto seri.

Gestione dei permessi

Permessi dei file

Quando utilizziamo:

```
ls -l
```

possiamo trovare una stringa simile a:

Questa stringa descrive il **tipo di file** e i relativi **permessi**.

È composta da 10 caratteri:

```
-rwxrwxrwx
├───┬───┬───
│   │   │
│   │   └─ altri utenti (others)
│   └───┬─ gruppo proprietario (group)
│       └─ proprietario (user/owner)
```

Il primo carattere indica il tipo di elemento:

```
- → file normale
d → directory
l → collegamento simbolico
```

I successivi nove caratteri sono divisi in tre gruppi:

The diagram illustrates the structure of permissions for three users (rwx, rwx, rwx) and their corresponding permissions for others, group, and owner. The permissions are organized into a tree structure where the root node is 'others', which branches into 'group' and 'owner'. The 'group' node further branches into three 'rwx' nodes, and the 'owner' node branches into three 'rwx' nodes.

Ogni gruppo può avere:

```
r → read      → lettura
w → write     → scrittura
x → execute   → esecuzione
```

Ad esempio:

significa:

```
owner  → rwx
group  → r-x
others → ---
```

Quindi:

- il proprietario può leggere, modificare ed eseguire;

- gli utenti del gruppo possono leggere ed eseguire;
- gli altri utenti non hanno alcun permesso.

Permessi delle directory

I permessi di una directory hanno un significato leggermente diverso rispetto a quelli di un file.

Esempio:

```
drwxr-x---
```

Il primo carattere:

```
d
```

indica che si tratta di una directory.

Anche qui abbiamo:

```
rwx rwx rwx
```

divisi tra:

- proprietario;
- gruppo;
- altri utenti.

Per una directory:

— Read

Permette di leggere l'elenco dei nomi presenti nella directory.

In altre parole, permette di vedere quali elementi sono presenti, se le altre condizioni di accesso lo consentono.

— Write

Permette di modificare il contenuto della directory, ad esempio creare, eliminare o rinominare elementi, purché siano rispettate anche le altre condizioni di accesso.

— Execute

Su una directory  significa principalmente poter attraversare/accedere alla directory e accedere agli elementi al suo interno quando i relativi permessi lo consentono.

Questo è un punto molto importante: `x` su una directory non significa semplicemente "eseguire la directory".

`ls -ld` — Visualizzare i permessi della directory stessa

Per visualizzare le informazioni della directory:

```
ls -ld nome_directory
```

L'opzione `-d` fa sì che `ls` mostri la directory stessa invece di elencarne il contenuto.

Chi può modificare i permessi?

Con `chmod`, normalmente può modificare i permessi:

- il proprietario del file;
- `root`;
- un utente che dispone dei privilegi amministrativi necessari.

Un normale utente non può modificare arbitrariamente i permessi dei file appartenenti a un altro utente.

`chmod` — Modificare i permessi

Il comando:

```
chmod
```

significa **change mode** e permette di modificare i permessi di file e directory.

Esistono principalmente due modalità:

- modalità simbolica;
- modalità ottale.

`chmod` — Modalità simbolica

È possibile aggiungere un permesso utilizzando:

```
chmod +x nome_file
```

Questo aggiunge il permesso `x` al file secondo il comportamento predefinito di `chmod`, normalmente applicandolo a user, group e others quando non viene specificato un destinatario.

È comunque più chiaro e consigliabile specificare esplicitamente il destinatario:

```
chmod u+x nome_file
```

→ aggiunge **x** al proprietario.

```
chmod g+x nome_file
```

→ aggiunge **x** al gruppo.

```
chmod o+x nome_file
```

→ aggiunge **x** agli altri utenti.

```
chmod a+x nome_file
```

→ aggiunge **x** a tutti.

Dove:

```
u → user/owner  
g → group  
o → others  
a → all
```

È possibile applicare più modifiche contemporaneamente:

```
chmod o+x,o+r nome_file
```

È equivalente a:

```
chmod o+rx nome_file
```

```
chmod o=rw nome_file
```

Il simbolo **=** permette di impostare i permessi esattamente come specificato, quindi eventuali permessi precedenti di **others** vengono rimossi.

Per rimuovere tutti i permessi da **others** :

```
chmod o= nome_file
```

Per rimuovere solo `x`:

```
chmod -x nome_file
```

Oppure, in modo esplicito:

```
chmod u-x nome_file  
chmod g-x nome_file  
chmod o-x nome_file
```

Per rimuoverlo da tutti:

```
chmod a-x nome_file
```

Permessi in formato ottale

I permessi possono essere rappresentati utilizzando numeri ottali.

I valori fondamentali sono:

Valore	Permessi	Significato
0	---	nessun permesso
1	--X	esecuzione
2	-W-	scrittura
3	-WX	scrittura + esecuzione
4	r--	lettura
5	r-X	lettura + esecuzione
6	rw-	lettura + scrittura
7	rwx	lettura + scrittura + esecuzione

Ogni cifra rappresenta uno dei tre gruppi:

```
chmod 754 file  
  |||  
  ||  └─ others  
  |   └─ group  
  └─  owner
```

Quindi:

```
7 → rwx  
5 → r-x  
4 → r--
```

e:

```
754 = rwxr-xr--
```

`chmod 777` — Tutti i permessi per tutti

Il comando:

```
chmod 777 nome_file
```

imposta:

```
owner  → rwx  
group  → rwx  
others → rwx
```

quindi:

```
rwxrwxrwx
```

Attenzione: `777` concede a tutti gli utenti del sistema la possibilità di leggere, modificare ed eseguire il file. È quindi un'impostazione da utilizzare con molta cautela, soprattutto su sistemi reali.

`chmod 710` — Permessi differenti per owner, group e others

Il comando:

```
chmod 710 nome_file
```

imposta:

```
owner  → rwx  
group  → --x  
others → ---
```

Quindi:

```
rwx--x---
```

Il proprietario ha tutti i permessi, il gruppo ha solamente `x` e gli altri utenti non hanno alcun permesso.

Bit speciali: SUID, SGID e Sticky Bit

Oltre ai normali permessi `rwX`, Linux dispone di tre **special permission bits**:

```
SUID
SGID
Sticky Bit
```

Sono tre bit speciali che modificano il comportamento dei normali permessi.

Nella notazione ottale vengono indicati con una quarta cifra, prima delle tre cifre dei permessi:

Esempio:

```
chmod 4755 file
      ^
      |
      bit speciale
```

Il 4 indica il **SUID**.

I valori principali sono:

```
4 → SUID
2 → SGID
1 → Sticky Bit
```

Si possono anche sommare:

```
4 + 2 = 6 → SUID + SGID
4 + 1 = 5 → SUID + Sticky Bit
2 + 1 = 3 → SGID + Sticky Bit
4 + 2 + 1 = 7 → tutti e tre
```

SUID — 4

SUID (Set User ID) è importante soprattutto sui **file eseguibili**.

Quando un eseguibile ha il SUID, viene eseguito con l'**UID del proprietario del file**, invece che con quello dell'utente che lo esegue.

Per esempio:

```
chmod 4755 file
```

Il 4 attiva il SUID.

Se il proprietario del file è `root`, il programma può essere eseguito con l'**effective UID di root**.

Per questo un file SUID appartenente a `root` è particolarmente importante in cybersecurity: una vulnerabilità nell'eseguibile potrebbe permettere di ottenere privilegi elevati.

Nel `ls -l`, il SUID appare come `s` nella posizione dell'`x` del proprietario:

```
-rwsr-xr-x
  ^
  SUID
```

Quindi:

```
x → esecuzione normale
s → esecuzione + SUID
```

SGID — 2

SGID (Set Group ID) cambia comportamento a seconda che sia applicato a un file o a una directory.

Su un **file eseguibile**, il programma viene eseguito con il **GID del gruppo proprietario del file**.

Per esempio:

```
-rwxr-sr-x
  ^
  SGID
```

Sulle **directory**, invece, lo SGID ha un significato molto importante:

I nuovi file e le nuove directory create al suo interno ereditano il gruppo della directory.

Esempio:

```
directory
gruppo = developers
SGID = attivo
```

Un utente crea:

```
directory/file.txt
```

Il file erediterà il gruppo `developers`.

Quindi lo SGID sulle directory è molto utile per creare directory condivise tra utenti dello stesso gruppo.

Sticky Bit — 1

Lo Sticky Bit è soprattutto significativo sulle **directory**.

Serve a impedire che un utente possa cancellare o rinominare liberamente i file appartenenti agli altri utenti all'interno di una directory condivisa.

L'esempio classico è:

```
/tmp
```

Una directory come `/tmp` può essere utilizzata da molti utenti, ma con Sticky Bit:

```
utente A → può cancellare i propri file
utente B → può cancellare i propri file
utente A → non può normalmente cancellare i file di B
```

Nel `ls -l` lo vedi come `t`:

```
drwxrwxrwt
      ^
    Sticky Bit
```

Da ricordare

```
4 → SUID
    "esegui come il proprietario"

2 → SGID
  file:  "esegui come il gruppo"
  directory: "eredita il gruppo"

1 → Sticky Bit
  directory: "proteggi i file degli altri utenti"
```

Quindi, quando vedi:

```
chmod 4755 file
```

devi leggerlo semplicemente come:

```
4    → SUID
755  → permessi normali
```

e:

```
-rwsr-xr-x
      ^
```

SUID attivo

La cosa più importante, in ottica cybersecurity, è quindi capire cosa succede quando un eseguibile ha SUID, soprattutto se il proprietario è `root`.

Permessi predefiniti dei nuovi file

Quando viene creato un nuovo file, i permessi iniziali dipendono dal programma che lo crea e dalla umask del processo.

Su molte distribuzioni, un normale file di testo creato dall'utente può risultare, ad esempio:

```
-rw-r--r--
```

ovvero:

```
owner  → rw-  
group → r--  
others → r--
```

Tuttavia, non bisogna considerare `-rw-r--r--` come un valore universale.

I permessi iniziali sono influenzati dalla `umask`.

Inoltre, i file normalmente non vengono creati con il bit di esecuzione impostato.

Visualizzare e leggere i file

`cat`

```
cat nome_file
```

visualizza il contenuto di un file.

È possibile utilizzare anche un percorso assoluto:

```
cat /etc/passwd
```

Per numerare le righe:

```
cat -n nome_file
```

`more`

```
more nome_file
```

visualizza il contenuto del file una schermata alla volta.

È utile soprattutto per file lunghi.

less

```
less nome_file
```

permette di visualizzare e navigare comodamente all'interno di file lunghi.

Durante la visualizzazione sono disponibili diversi comandi:

- `Space` → pagina successiva;
- `b` → pagina precedente;
- frecce → spostamento;
- `/testo` → ricerca;
- `n` → risultato successivo della ricerca;
- `q` → uscita;
- `h` → visualizza l'help.

`less` è particolarmente utile per analizzare log e file di grandi dimensioni.

head — Inizio di un file

```
head nome_file
```

mostra le prime 10 righe del file.

Per specificare il numero di righe:

```
head -n 5 nome_file
```

oppure:

```
head -5 nome_file
```

Con più file:

```
head -n 5 *.log
```

mostra le prime 5 righe di ogni file `.log`.

Con:

```
head -q -n 5 *.log
```

vengono nascosti gli header che indicano il nome dei file.

tail — Fine di un file

```
tail nome_file
```

mostra le ultime 10 righe.

Per specificare il numero:

```
tail -n 5 nome_file
```

Una funzione molto importante in cybersecurity e amministrazione di sistema è:

```
tail -f /var/log/syslog
```

che permette di seguire in tempo reale le nuove righe aggiunte al file.

cut — Estrarre parti di testo

`cut` permette di estrarre caratteri o campi da ogni riga.

Selezionare caratteri

```
cut -c 5-10 file.txt
```

mostra i caratteri dalla posizione 5 alla posizione 10 di ogni riga.

```
cut -c 5,10 file.txt
```

mostra solamente il quinto e il decimo carattere.

```
cut -c 1 file.txt
```

mostra il primo carattere di ogni riga.

Selezionare campi

Con `-d` si specifica il delimitatore.

Per esempio, `/etc/passwd` utilizza `:` come separatore:

```
cut -d ":" -f 1 /etc/passwd
```

Significa:

- `d ":"` → usa `:` come delimitatore;
- `f 1` → estrai il primo campo.

Il risultato sarà l'elenco degli username.

5. Ricerca e manipolazione del testo

`find` — Cercare file e directory

Il comando:

```
find
```

permette di cercare file e directory all'interno di una gerarchia del filesystem.

Esempio:

```
find .
```

cerca ricorsivamente a partire dalla directory corrente.

`find` attraversa quindi normalmente anche le sottodirectory.

`find -maxdepth` — Limitare la profondità della ricerca

Per evitare di scendere ricorsivamente in tutta la struttura possiamo utilizzare:

```
find . -maxdepth 1
```

Questo limita la ricerca al livello iniziale.

Ad esempio:

```
find . -maxdepth 2
```

permette di includere anche il livello immediatamente successivo.

È più corretto pensare a `-maxdepth` come al numero massimo di livelli della gerarchia visitati, considerando il punto di partenza come livello `1`.

`find -name` — Cercare per nome

Per cercare un file con un determinato nome:

```
find . -name "file.txt"
```

Per cercare tutti i file `.txt`:

```
find . -name "*.txt"
```

Le virgolette sono importanti perché impediscono alla shell di espandere `*.txt` prima che venga eseguito `find`.

`find -iname` — Ricerca senza distinzione tra maiuscole e minuscole

L'opzione:

```
-iname
```

funziona come `-name`, ma non distingue tra maiuscole e minuscole.

Esempio:

```
find . -iname "*.txt"
```

può trovare:

```
file.txt  
FILE.TXT  
File.Txt
```

`find -type` — Cercare per tipo

L'opzione:

```
-type
```

permette di specificare il tipo di elemento da cercare.

I valori più utilizzati sono:

```
f → file normale  
d → directory  
l → collegamento simbolico
```

Esempio:

```
find . -type f
```

cerca solamente i file normali.

Per cercare directory:

```
find . -type d
```

Per cercare symlink:

```
find . -type l
```

find -mtime e **find -mmin** — Cercare in base alla modifica

find può cercare file in base al momento dell'ultima modifica.

mtime

Utilizza giorni:

```
find . -mtime -1
```

cerca elementi modificati nelle ultime 24 ore circa.

mmin

Utilizza minuti:

```
find . -mmin -30
```

cerca elementi modificati negli ultimi 30 minuti.

I simboli hanno un significato importante:

```
- → meno di  
+ → più di
```

find -user — Cercare file appartenenti a un utente

Esempio:

```
find /home -user mario
```

cerca i file e le directory appartenenti all'utente `mario`.

find -group — Cercare file appartenenti a un gruppo

Esempio:

```
find /home -group pentest
```

cerca gli elementi appartenenti al gruppo `pentest`.

find -size — Cercare in base alla dimensione

Esempio:

```
find . -type f -size +2M
```

cerca file normali con dimensione superiore a 2 MiB secondo la semantica di `find`.

Altri esempi:

```
find . -type f -size 2M
```

→ dimensione nell'intervallo corrispondente a 2 MiB secondo la modalità di confronto di `find`.

```
find . -type f -size -2M
```

→ dimensione inferiore a 2 MiB.

Per confronti di dimensione molto precisi, è importante conoscere la semantica delle unità utilizzate da `find`.

find -perm — Cercare in base ai permessi

L'opzione:

```
-perm
```

permette di cercare elementi in base alla loro modalità.

Per cercare esattamente i permessi `777`:

```
find . -type f -perm 0777
```

Per cercare file che hanno almeno quei permessi:

```
find . -type f -perm -0777
```

Per cercare file che hanno uno o più dei bit specificati:

```
find . -type f -perm /0777
```

Queste tre forme non sono equivalenti:

```
-perm 0777 → corrispondenza esatta  
-perm -0777 → tutti i bit indicati devono essere presenti  
-perm /0777 → almeno uno dei bit indicati deve essere presente
```

Questo è particolarmente importante in cybersecurity: cercare `-perm 0777` non significa "trova tutti i file con permessi troppo permissivi". Trova solamente quelli con modalità esattamente `0777`.

`find -exec` — Eseguire un comando sui risultati

L'opzione:

```
-exec
```

permette di eseguire un comando per ogni elemento trovato.

Esempio:

```
find . -type f -name "*.txt" -exec ls -l {} \;
```

Qui:

```
{}
```

rappresenta l'elemento trovato da `find`.

Mentre:

```
\;
```

indica la fine del comando passato a `-exec`.

Quindi, concettualmente:

```
find trova un file
  ↓
{} viene sostituito con il percorso del file
  ↓
viene eseguito ls -l sul file
```

Una forma spesso più efficiente, quando compatibile con il comando utilizzato, è:

```
find . -type f -name "*.txt" -exec ls -l {} +
```

In questo caso `find` può passare più risultati allo stesso comando invece di eseguire un comando separato per ogni file.

Esempio completo di `find`

Supponiamo di voler cercare, dalla directory corrente:

- file normali;
- con estensione `.txt`;
- senza distinzione tra maiuscole e minuscole;
- con dimensione superiore a 2 MiB;
- appartenenti all'utente `mario`;
- con permessi esattamente `0777`.

Possiamo utilizzare:

```
find . -type f -iname "*.txt" -size +2M -perm 0777 -user mario
```

Le condizioni vengono combinate come **AND**, quindi un elemento deve soddisfare tutte le condizioni specificate.

Se invece vogliamo cercare file che abbiano **almeno** i bit di `600`:

```
find . -type f -perm -600
```

Questo non significa necessariamente che il file abbia esattamente `600` : significa che i bit indicati da `600` sono presenti.

Perché `find` è importante in cybersecurity

`find` è particolarmente utile durante l'enumerazione di un sistema Linux.

Può essere utilizzato, ad esempio, per individuare:

- file appartenenti a un determinato utente;
- file appartenenti a un determinato gruppo;
- file con permessi particolari;
- file modificati recentemente;
- file di una determinata dimensione;
- directory specifiche;
- collegamenti simbolici;
- file con permessi speciali come SUID/SGID.

Per esempio, per cercare file con SUID impostato:

```
find / -type f -perm -4000 2>/dev/null
```

Qui:

```
/           → parte dalla root del filesystem
-type f     → cerca file normali
-perm -4000 → richiede il bit SUID
2>/dev/null → nasconde gli errori di permesso
```

Questo tipo di ricerca è particolarmente importante durante l'**enumerazione dei privilegi** in Linux.

`grep` — Ricerca di testo

`grep` è uno degli strumenti più importanti per l'analisi di file e log.

```
grep "testo" file.txt
```

mostra le righe che contengono `testo`.

`grep -v`

```
grep -v "testo" file.txt
```

mostra le righe che **non** contengono `testo`.

`grep -n`

```
grep -n "testo" file.txt
```

mostra anche il numero della riga.

`grep -c`

```
grep -c "testo" file.txt
```

conta quante righe corrispondono alla ricerca.

`-c` conta le righe che fanno match, non necessariamente il numero totale di occorrenze della parola.

`grep -r`

```
grep -r "password" /etc
```

esegue una ricerca ricorsiva nelle directory e sottodirectory.

È molto utile durante l'analisi di configurazioni e file di sistema.

`grep -i`

```
grep -i "password" file.txt
```

ignora la differenza tra maiuscole e minuscole.

Quindi può trovare:

```
password
Password
PASSWORD
PaSsWoRd
```

`grep -w`

```
grep -w "testo" file.txt
```

cerca `testo` come parola intera.

Per esempio, con:

```
testo
testone
mio testo
```

`grep -w "testo"` trova le righe contenenti la parola `testo`, ma non considera `testone` come corrispondenza della parola intera.

`grep -l`

Per visualizzare solamente i nomi dei file che contengono una determinata stringa:

```
grep -l "password" *.txt
```

- `l` significa "list filenames".

`sort` — Ordinare le righe

```
sort file.txt
```

ordina le righe in ordine lessicografico.

Per invertire l'ordine:

```
sort -r file.txt
```

Per ignorare maiuscole e minuscole:

```
sort -f file.txt
```

Per eliminare i duplicati durante l'ordinamento:

```
sort -u file.txt
```

Ordinamento numerico

Il normale:

```
sort file.txt
```

non interpreta necessariamente i valori come numeri.

Per esempio:

```
0
1190
2020
4
80
```

può essere ordinato lessicograficamente.

Con:

```
sort -n file.txt
```

i valori vengono interpretati numericamente:

```
0
4
80
1190
2020
```

uniq — Gestire righe duplicate

```
uniq file.txt
```

rimuove le righe duplicate **adiacenti**.

Questo è importante:

```
ciao
ciao
mondo
ciao
```

con:

```
uniq file.txt
```

diventa:

```
ciao
mondo
ciao
```

Per eliminare tutti i duplicati, indipendentemente dalla loro posizione, è comune utilizzare:

```
sort file.txt | uniq
```

oppure:

```
sort -u file.txt
```

uniq -d

```
uniq -d file.txt
```

mostra solamente le righe duplicate adiacenti.

uniq -c

```
uniq -c file.txt
```

mostra quante volte ogni riga compare consecutivamente.

Esempio:

```
3 ciao  
1 mondo  
2 test
```

uniq -i

```
uniq -i file.txt
```

ignora la differenza tra maiuscole e minuscole durante il confronto.

Non significa "mostra solamente gli elementi con la maiuscola".

wc — Contare righe, parole e caratteri

```
wc file.txt
```

mostra:

- numero di righe;

- numero di parole;
- numero di byte.

Opzioni importanti:

```
wc -l file.txt
```

numero di righe.

```
wc -w file.txt
```

numero di parole.

```
wc -m file.txt
```

numero di caratteri.

```
wc -c file.txt
```

numero di byte.

rev — Invertire le righe

```
rev file.txt
```

inverte i caratteri di ogni riga.

Per esempio:

```
ciao
```

diventa:

```
oaic
```

tr — Trasformare caratteri

tr legge lo standard input e sostituisce, elimina o comprime caratteri.

Sostituzione semplice

```
echo "casa" | tr 'a' 'e'
```

Output:

```
cese
```

Sostituire più caratteri

```
echo "casa" | tr 'ac' 'en'
```

a viene sostituita con **e**.

c viene sostituita con **n**.

Output:

```
nese
```

tr -d — Eliminare caratteri

```
echo "ciao" | tr -d 'a'
```

Output:

```
cio
```

tr -s — Comprimere caratteri consecutivi

```
echo "ciaao" | tr -s 'o'
```

Output:

```
ciao
```

- **s** comprime sequenze consecutive dello stesso carattere.

Classi di caratteri

È possibile utilizzare classi POSIX:

```
[[:digit:]]
```

numeri.

```
[[:alpha:]]
```

lettere.

```
[[:lower:]]
```

lettere minuscole.

```
[[:upper:]]
```

lettere maiuscole.

```
[[:space:]]
```

spazi e caratteri di spazio.

Esempio:

```
echo "Ciao 123" | tr '[:lower:]' '[:upper:]'
```

Output:

```
CIAO 123
```

Per eliminare tutti i numeri:

```
tr -d '[:digit:]' < file.txt
```

sed — Stream Editor

sed è uno strumento per elaborare e modificare testo in modo programmabile.

La sintassi generale può essere rappresentata come:

```
sed [opzioni] 'istruzione' file
```

sed -n

Normalmente `sed` stampa ogni riga elaborata.

Con:

```
sed -n 'lp' file.txt
```

- `n` disabilita la stampa automatica e `lp` dice di stampare solamente la prima riga.

Stampare una riga

```
sed -n 'lp' file.txt
```

mostra solamente la prima riga.

Per stampare le righe da 1 a 3:

```
sed -n '1,3p' file.txt
```

Eliminare righe dall'output

```
sed '1,3d' file.txt
```

elimina dall'output le righe dalla 1 alla 3.

Per impostazione predefinita, `sed` **non modifica il file originale**. Modifica il flusso di output.

Sostituire testo

```
sed 's/ciao/CIA0/' file.txt
```

sostituisce la prima occorrenza di `ciao` in ogni riga.

Per sostituire tutte le occorrenze nella riga:

```
sed 's/ciao/CIA0/g' file.txt
```

`g` significa "global".

Sostituire un'intera riga

```
sed '/a/c\CIAO' file.txt
```

sostituisce con `CIAO` ogni riga che contiene `a`.

Modificare realmente il file

Senza opzioni particolari:

```
sed 's/vecchio/nuovo/g' file.txt
```

mostra solamente il risultato.

Per modificare direttamente il file, su GNU `sed` :

```
sed -i 's/vecchio/nuovo/g' file.txt
```

Prima di usare `-i` su file importanti è consigliabile fare una copia di backup.

`cmp` — Confronto byte per byte

```
cmp file1 file2
```

confronta due file byte per byte e normalmente segnala la prima differenza.

```
cmp -b file1 file2
```

fornisce informazioni aggiuntive sul byte differente.

`diff` — Confrontare file

```
diff file1 file2
```

mostra le differenze tra i due file.

In un output tradizionale di `diff` :

- `<` indica contenuto proveniente dal primo file;
- `>` indica contenuto proveniente dal secondo file.

```
diff -q
```

```
diff -q file1 file2
```

non mostra le differenze dettagliate, ma indica solamente se i file differiscono.

diff -i

```
diff -i file1 file2
```

ignora le differenze tra maiuscole e minuscole.

diff -b

```
diff -b file1 file2
```

ignora le differenze nella quantità di spazi bianchi.

patch — Applicare un file di differenze

È possibile generare un file contenente le differenze:

```
diff file_originale file_modificato > modifiche.patch
```

Successivamente è possibile applicare le modifiche:

```
patch file_originale < modifiche.patch
```

In questo modo **patch** applica al file originale le modifiche descritte nel file **.patch**.

| L'ordine dei file nel comando **diff** è importante: determina la direzione delle modifiche.

6. Output, input e redirezioni

echo

Il comando **echo** stampa del testo o il contenuto di una variabile sullo standard output.

```
echo "Ciao"
```

Output:

```
Ciao
```

È possibile utilizzare le sequenze di escape con `-e`:

```
echo -e "Riga 1\nRiga 2"
```

Output:

```
Riga 1  
Riga 2
```

`\n` rappresenta un ritorno a capo.

Per inserire una tabulazione:

```
echo -e "Nome:\tMario"
```

`\t` rappresenta una tabulazione.

Il comportamento di `echo -e` può variare tra implementazioni. Per output complessi, `printf` è generalmente più prevedibile.

Variabili con `echo`

```
echo $USER
```

mostra il nome dell'utente corrente.

```
echo $PATH
```

mostra il contenuto della variabile `PATH`.

`PATH` contiene una serie di directory nelle quali la shell cerca i programmi quando vengono eseguiti senza specificare il percorso completo.

Per esempio:

```
echo $PATH
```

potrebbe restituire:

```
/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/sbin
```

Le directory sono separate dal carattere `:`.

Apici singoli e doppi

Gli apici singoli:

```
echo '$PATH'
```

impediscono l'espansione della variabile.

Output:

```
$PATH
```

Gli apici doppi:

```
echo "$PATH"
```

permettono invece alla shell di espandere la variabile.

Espansione aritmetica

Per eseguire calcoli in Bash:

```
echo $((50 / 25))
```

Output:

```
2
```

Gli spazi all'interno dell'espressione sono normalmente consentiti:

```
echo $((50 + 25))
```

Le doppie parentesi `(( ))` vengono utilizzate da Bash per l'aritmetica.

Redirezione dell'output

La shell permette di modificare la destinazione dell'output di un comando.

```
>
```

```
echo "ciao" > file.txt
```

crea `file.txt` se non esiste e scrive `ciao`.

Se il file esiste già, **lo sovrascrive**.

>>

```
echo "seconda riga" >> file.txt
```

aggiunge il testo alla fine del file senza cancellare il contenuto precedente.

2>

Il file descriptor `2` rappresenta lo standard error.

```
comando 2> errori.txt
```

redireziona gli errori nel file.

&>

```
comando &> output.txt
```

redireziona sia:

- standard output (`1`);
- standard error (`2`).

nel file indicato.

Un'alternativa molto utilizzata è:

```
comando > output.txt 2>&1
```

`/dev/null`

```
/dev/null
```

è un dispositivo speciale che scarta qualsiasi dato ricevuto.

Per esempio:

```
comando 2> /dev/null
```

scarta gli errori.

Esempio:

```
cp test3 ./copia 2> /dev/null
```

Se `cp` produce un errore, l'errore non viene mostrato sul terminale.

`/dev/null` non "salva" i dati: li scarta.

; — Eseguire più comandi

Il punto e virgola permette di eseguire più comandi consecutivamente:

```
comando1 ; comando2 ; comando3
```

Il comando successivo viene eseguito indipendentemente dal successo o dal fallimento del precedente.

Per esempio:

```
mkdir test ; cd test
```

| — Pipe

La pipe:

```
|
```

collega lo standard output di un comando allo standard input del comando successivo.

Esempio:

```
cat file.txt | grep "password"
```

Il risultato di `cat` viene utilizzato come input di `grep`.

Un esempio molto comune:

```
ps aux | grep firefox
```

In questo caso l'output di `ps aux` viene passato a `grep`.

La pipe non significa semplicemente "mostra solo l'ultimo comando": collega due flussi di dati.

Espressioni regolari — Regex

Le espressioni regolari permettono di descrivere pattern di testo.

Sono molto utilizzate con:

- `grep` ;
- `sed` ;
- `awk` ;
- strumenti di analisi dei log;
- cybersecurity e digital forensics.

^ — Inizio della riga

```
grep '^a' file.txt
```

trova le righe che iniziano con `a`.

\$ — Fine della riga

```
grep 'a$' file.txt
```

trova le righe che terminano con `a`.

. — Un carattere qualsiasi

```
grep 'c..a' file.txt
```

cerca pattern costituiti da:

```
c + carattere qualsiasi + carattere qualsiasi + a
```

[] — Insieme di caratteri

```
grep 'c[aei]sa' file.txt
```

cerca:

```
casa  
cesa  
cisa
```

perché nella posizione indicata può esserci **a**, **e** oppure **i**.

[^] — Negazione

```
grep 'c[^aei]s' file.txt
```

cerca una **c**, seguita da un carattere che **non** appartiene a **aei**, seguito da **s**.

grep -E — Extended Regular Expressions

Per utilizzare le espressioni regolari estese:

```
grep -E 'pattern' file.txt
```

{n} — Esattamente n occorrenze

```
grep -E '^[h-q]{8}$' file.txt
```

cerca righe composte esattamente da 8 caratteri, tutti appartenenti all'intervallo **h-q**.

{n,} — Almeno n occorrenze

```
grep -E '^[h-q]{8,}$' file.txt
```

richiede almeno 8 caratteri.

{,n} — Da zero a n occorrenze

```
grep -E '^[h-q]{,8}$' file.txt
```

su GNU grep indica un massimo di 8 occorrenze.

.* — Qualsiasi sequenza di caratteri

```
grep -E '^a.*t$' file.txt
```

trova righe che iniziano con `a` e terminano con `t`.

Alternativa `|`

```
grep -E '^a.*t$|^e.*x$' file.txt
```

cerca righe che soddisfano una delle due condizioni:

- iniziano con `a` e terminano con `t`;
- iniziano con `e` e terminano con `x`.

```
grep -E '(c[aeiou]){2}' file.txt
```

cerca due ripetizioni consecutive del pattern:

c + una vocale

Per esempio:

```
cacic...
```

Il quantificatore `{2}` si applica all'intero gruppo.

Exit status

Ogni comando restituisce un valore numerico chiamato **exit status**.

Il valore:

0	successo
1-255	errore o altro risultato

Per visualizzare l'exit status dell'ultimo comando:

```
echo $?
```

Esempio:

```
ls /tmp  
echo $?
```

Se `ls` ha avuto successo, normalmente:

```
0
```

Se invece il comando fallisce:

```
ls /directory/inesistente  
echo $?
```

potremmo ottenere un valore diverso da `0`.

Non bisogna interpretare ogni valore diverso da zero come "errore generico": il significato preciso dipende dal programma.

Un valore molto comune è:

```
127
```

che Bash utilizza generalmente quando un comando non viene trovato.

Questo meccanismo è fondamentale negli script perché permette di prendere decisioni in base all'esito dei comandi:

```
if comando; then  
    echo "Comando riuscito"  
else  
    echo "Comando fallito"  
fi
```

In questo caso non stiamo controllando direttamente il testo prodotto dal comando: stiamo controllando il suo **exit status**.

7. Shell scripting

Uno **shell script** è un file di testo contenente una sequenza di comandi che possono essere eseguiti dalla shell.

Un esempio:

```
#!/bin/bash  
  
echo "Ciao"  
pwd  
whoami
```

La prima riga:

```
#!/bin/bash
```

viene chiamata **shebang**.

Indica quale interprete deve essere utilizzato quando lo script viene eseguito direttamente.

Un'alternativa più portatile può essere:

```
#!/usr/bin/env bash
```

che cerca `bash` nelle directory definite dal `PATH`.

Il file può essere salvato come:

```
script.sh
```

Permesso di esecuzione

Per eseguire direttamente uno script è necessario che il file abbia il permesso di esecuzione:

```
chmod +x script.sh
```

Successivamente:

```
./script.sh
```

Il prefisso `./` indica che lo script si trova nella directory corrente.

Il fatto che un file sia chiamato `.sh` non gli assegna automaticamente il permesso di esecuzione.

Modi per eseguire uno script

```
./script.sh
```

```
./script.sh
```

richiede il permesso di esecuzione e utilizza l'interprete indicato dallo shebang.

```
bash script.sh
```

```
bash script.sh
```

esegue lo script tramite Bash.

In questo caso il permesso di esecuzione del file non è necessario.

```
source script.sh
```

```
source script.sh
```

esegue lo script nella shell corrente.

È equivalente a:

```
. script.sh
```

Questa differenza è importante perché variabili, alias, funzioni e modifiche all'ambiente effettuate dallo script possono rimanere nella shell corrente.

Commenti negli script

Un commento in Bash inizia con `#`.

```
# Questo è un commento  
echo "Ciao"
```

Bash ignora tutto ciò che si trova dopo `#` sulla stessa riga, salvo casi particolari in cui il carattere viene utilizzato all'interno di stringhe o altri contesti sintattici.

Variabili Bash

Una variabile può essere assegnata senza spazi intorno a `=`:

```
nome="Mario"
```

Errato:

```
nome = "Mario"
```

In Bash `=` nell'assegnazione non è un operatore di confronto.

Utilizzare una variabile

Per accedere al valore:

```
echo $nome
```

È spesso preferibile utilizzare le parentesi graffe:

```
echo ${nome}
```

Le parentesi graffe sono particolarmente utili quando il nome della variabile è seguito immediatamente da altri caratteri.

Esempio:

```
nome="Mario"
echo "${nome}l"
```

produce:

```
Mariol
```

Variabili predefinite

Bash mette a disposizione numerose variabili speciali.

\$0

Indica il nome dello script o del comando con cui è stata invocata la shell.

In uno script:

```
echo "$0"
```

può restituire:

```
./script.sh
```

\$\$

```
echo $$
```

restituisce il PID della shell corrente.

\$?

```
echo $?
```

restituisce l'exit status dell'ultimo comando eseguito.

\$#

```
echo $#
```

indica il numero di parametri passati allo script.

\$@

```
echo "$@"
```

rappresenta i parametri passati allo script.

È particolarmente utile quando si vogliono elaborare i parametri mantenendoli separati.

\$*

```
echo "$*"
```

rappresenta anch'esso i parametri, ma il comportamento con le virgolette è diverso da **"\$@"**.

Per lo scripting Bash, quando si vogliono elaborare i parametri uno per uno, normalmente si preferisce:

```
"$@"
```

\$RANDOM

Bash mette a disposizione:

```
$RANDOM
```

che restituisce un numero pseudocasuale.

Esempio:

```
echo $RANDOM
```

Per ottenere un valore tra 0 e 25 :

```
echo $((RANDOM % 26))
```

Per ottenere un numero tra 1 e 100:

```
echo $((RANDOM % 100 + 1))
```

`$RANDOM` è adatto a semplici esigenze di scripting, ma non deve essere utilizzato per generare valori casuali crittograficamente sicuri, ad esempio password o chiavi.

Tabella delle variabili speciali principali

Variabile	Significato
<code>\$0</code>	Nome/percorso con cui è stato invocato lo script
<code>\$1</code> , <code>\$2</code> , ...	Parametri posizionali
<code>\$#</code>	Numero dei parametri
<code>\$@</code>	Tutti i parametri
<code>\$*</code>	Tutti i parametri
<code>\$?</code>	Exit status dell'ultimo comando
<code>\$\$</code>	PID della shell corrente
<code>\$_</code>	PID dell'ultimo processo avviato in background
<code>\$RANDOM</code>	Numero pseudo-casuale
<code>\$HOME</code>	Home dell'utente
<code>\$USER</code>	Nome dell'utente
<code>\$PATH</code>	Directory utilizzate per cercare gli eseguibili
<code>\$SHELL</code>	Shell configurata per l'utente

`$@` e `$*` non sono perfettamente equivalenti quando vengono utilizzati tra virgolette. Negli script, quando si vogliono trattare correttamente i parametri individualmente, è generalmente preferibile

`"$@"`.

Parametri dello script

Supponiamo di avere:

```
#!/bin/bash

echo "Nome: $1"
echo "Secondo parametro: $2"
```

Eseguiamo:

```
./script.sh Mario Rossi
```

Output:

```
Nome: Mario  
Secondo parametro: Rossi
```

`$1` è il primo parametro.

`$2` è il secondo parametro.

E così via.

Parametri contenenti spazi

Se vogliamo passare un singolo parametro contenente spazi:

```
./script.sh "Mario Rossi"
```

In questo caso:

```
$1
```

contiene:

```
Mario Rossi
```

senza essere diviso in due parametri.

read — Leggere input dell'utente

Il comando:

```
read nome
```

attende che l'utente inserisca un valore.

Esempio:

```
echo "Inserisci il tuo nome:"  
read nome  
echo "Ciao $nome"
```

È possibile utilizzare `-p`:

```
read -p "Inserisci il tuo nome: " nome
```

In questo modo il prompt viene mostrato direttamente da `read`.

`echo -n`

```
echo -n "Inserisci nome: "  
read nome
```

- `n` impedisce a `echo` di andare automaticamente a capo.

Array Bash

Per creare un array:

```
nome_array=("dato1" "dato2" "dato3")
```

Gli elementi sono indicizzati a partire da `0`.

Accedere a un elemento

```
echo "${nome_array[0]}"
```

Output:

```
dato1
```

Secondo elemento:

```
echo "${nome_array[1]}"
```

Tutti gli elementi

```
echo "${nome_array[@]}"
```

oppure:

```
echo "${nome_array[*]}"
```

Le due forme hanno comportamenti differenti soprattutto quando vengono utilizzate tra virgolette.
Per iterare sugli elementi separatamente è generalmente preferibile:

```
"${nome_array[@]}"
```

Numero di elementi

```
echo "${#nome_array[@]}"
```

restituisce il numero di elementi presenti nell'array.

Indici dell'array

```
echo "${!nome_array[@]}"
```

mostra gli indici presenti.

Eliminare un elemento

```
unset 'nome_array[0]'
```

elimina l'elemento con indice `0`.

| L'eliminazione di un elemento non necessariamente rinumeri gli altri indici.

Condizioni

if

La struttura fondamentale è:

```
if [ condizione ]; then
    comando
fi
```

Esempio:

```
if [ "$a" -eq "$b" ]; then
    echo "I numeri sono uguali"
fi
```

`if` esegue il blocco solo se la condizione restituisce exit status 0.

`if ... else`

È possibile specificare cosa fare quando la condizione è falsa:

```
if [ "$a" -eq "$b" ]; then
    echo "Uguali"
else
    echo "Diversi"
fi
```

`if ... elif ... else`

Per controllare più condizioni:

```
if [ "$a" -gt 10 ]; then
    echo "Maggiore di 10"
elif [ "$a" -eq 10 ]; then
    echo "Uguale a 10"
else
    echo "Minore di 10"
fi
```

Operatori logici

È possibile combinare più condizioni.

AND

Con `&&` entrambe le condizioni devono essere vere:

```
if [ "$eta" -ge 18 ] && [ "$patente" = "si" ]; then
    echo "Condizione verificata"
fi
```

È possibile utilizzare anche:

```
if [[ "$eta" -ge 18 && "$patente" = "si" ]]; then
    echo "Condizione verificata"
fi
```

`[[ ]]` è una costruzione interna di Bash e offre funzionalità più sicure e potenti rispetto a `[ ]`.

OR

Con `||` è sufficiente che almeno una condizione sia vera:

```
if [ "$utente" = "root" ] || [ "$utente" = "admin" ]; then
    echo "Utente amministrativo"
fi
```

NOT

`!` nega una condizione:

```
if [ ! -f "file.txt" ]; then
    echo "Il file non esiste"
fi
```

Operatori di confronto

```
-eq    uguale
-ne    diverso
-gt    maggiore
-ge    maggiore o uguale
-lt    minore
-le    minore o uguale
```

case

`case` è utile quando dobbiamo confrontare una variabile con diversi possibili valori.

Struttura:

```
case "$variabile" in
    valore1)
        comando
        ;;
    valore2)
        comando
        ;;
    *)
        comando
        ;;
esac
```

Esempio:

```
scelta="a"
```

```
case "$scelta" in
  a)
    echo "Hai scelto A"
    ;;
  b)
    echo "Hai scelto B"
    ;;
  *)
    echo "Scelta non valida"
    ;;
esac
```

- rappresenta il caso predefinito.

Cicli

Ciclo **for**

Il ciclo **for** permette di ripetere un'operazione per più elementi.

Esempio:

```
for nome in Mario Luigi Anna
do
  echo "$nome"
done
```

Output:

```
Mario
Luigi
Anna
```

È possibile scriverlo anche su una sola riga:

```
for nome in Mario Luigi Anna; do echo "$nome"; done
```

for con file

Esempio:

```
for file in *.txt
do
  echo "$file"
done
```

Il carattere ***** viene espanso dalla shell e permette di ottenere i file **.txt** presenti nella directory.

Ciclo **while**

while esegue ripetutamente un blocco finché la condizione rimane vera.

```
contatore=1

while [ "$contatore" -le 5 ]
do
    echo "$contatore"
    contatore=$((contatore + 1))
done
```

Output:

```
1
2
3
4
5
```

Ciclo **until**

until è simile a **while**, ma continua a eseguire il blocco finché la condizione è falsa.

```
contatore=1

until [ "$contatore" -gt 5 ]
do
    echo "$contatore"
    contatore=$((contatore + 1))
done
```

break

break interrompe immediatamente il ciclo.

```
for numero in 1 2 3 4 5
do
    if [ "$numero" -eq 3 ]; then
        break
    fi

    echo "$numero"
done
```

Output:

```
1
2
```

continue

continue salta l'iterazione corrente e passa alla successiva.

```
for numero in 1 2 3 4 5
do
    if [ "$numero" -eq 3 ]; then
        continue
    fi
    echo "$numero"
done
```

Output:

```
1
2
4
5
```

8. Processi

Un **processo** è un programma in esecuzione.

Quando un programma viene avviato, il sistema operativo crea una struttura dati associata al processo che contiene informazioni necessarie per gestirlo.

Tra queste informazioni troviamo:

- PID;
- PPID;
- stato;
- priorità;
- informazioni sull'utente;
- memoria utilizzata;
- file descriptor;
- directory di lavoro;
- segnali pendenti;
- informazioni necessarie per il cambio di contesto.

PID — Process ID

Il **PID** identifica un processo.

Per esempio:

```
echo $$
```

mostra il PID della shell corrente.

I PID vengono assegnati dal kernel e possono essere riutilizzati dopo che un processo termina.

PID 1

Il processo con PID:

```
1
```

è il processo iniziale dello spazio utente.

Nei sistemi moderni Linux basati su `systemd`, normalmente è:

```
systemd
```

In altri sistemi può essere presente un altro processo di inizializzazione.

Il PID 1 svolge funzioni particolari nella gestione dei processi e nell'adozione dei processi orfani.

PPID — Parent Process ID

Il **PPID** identifica il processo padre.

Per esempio, se un processo Bash avvia un programma:

```
Bash
└─ programma
```

il programma sarà un processo figlio della Bash e il suo PPID sarà il PID della Bash.

ps — Visualizzare i processi

```
ps
```

mostra i processi associati alla sessione corrente.

Un output tipico comprende:

PID	TTY	TIME	CMD
-----	-----	------	-----

- `PID` → identificativo del processo;

- `TTY` → terminale associato;
 - `TIME` → tempo CPU utilizzato;
 - `CMD` → comando che ha avviato il processo.
-

`ps -e`

```
ps -e
```

mostra i processi presenti nel sistema.

Una forma molto utilizzata è:

```
ps -ef
```

che mostra una visualizzazione più completa dei processi.

`ps -u`

```
ps -u nome_utente
```

mostra i processi associati a un determinato utente.

`ps -p`

```
ps -p 1234
```

mostra informazioni sul processo con PID `1234`.

È possibile specificare più PID:

```
ps -p 1234,5678
```

`ps -o`

Permette di scegliere quali colonne visualizzare:

```
ps -o pid,uid,pri,cmd
```

Per esempio:

- `pid` → PID;

- `uid` → User ID;
- `pri` → priorità;
- `cmd` → comando.

Gli spazi non sono generalmente necessari dopo le virgole.

`top` e `htop`

```
top
```

mostra in tempo reale informazioni sui processi.

Permette di osservare:

- CPU;
- memoria;
- PID;
- utenti;
- stato dei processi;
- carico del sistema.

`htop` offre un'interfaccia interattiva più comoda, quando installato:

```
htop
```

Per uscire:

```
q
```

Stati dei processi

I processi possono trovarsi in diversi stati.

I principali codici mostrati da `ps` includono:

```
R
```

Running/Runnable: il processo è in esecuzione o pronto per essere eseguito.

```
S
```

Interruptible sleep: il processo sta aspettando un evento e può normalmente essere risvegliato da un segnale.

D

Uninterruptible sleep: il processo è in attesa non interrompibile, tipicamente durante alcune operazioni di I/O.

T

Stopped: il processo è stato fermato.

Z

Zombie: il processo è terminato ma il processo padre non ha ancora raccolto il suo exit status.

I

Idle: stato utilizzato in particolari contesti del kernel, per esempio per alcuni thread del kernel.

R viene spesso descritto semplicemente come "running", ma in Linux può indicare anche un processo runnable, cioè pronto per essere eseguito.

Processo zombie

Quando un processo termina, il kernel conserva temporaneamente alcune informazioni sul processo, tra cui l'exit status.

Il processo rimane in stato:

Z

finché il processo padre non raccoglie il risultato tramite una funzione della famiglia `wait()`.

Dopo che il padre ha raccolto l'exit status, il processo viene completamente rimosso.

Priorità dei processi

La priorità influenza lo scheduling dei processi.

Linux utilizza diversi valori e meccanismi di scheduling; per i normali processi si incontrano anche i concetti di:

```
nice
```

e:

```
niceness
```

In generale, un valore di nice più alto indica una minore priorità relativa rispetto agli altri processi della stessa classe di scheduling.

È importante distinguere:

- **PRI** → priorità calcolata/utilizzata dal kernel;
- **NI** → valore di niceness.

Foreground e background

Normalmente un comando eseguito dal terminale viene eseguito in **foreground**.

Esempio:

```
cat
```

Il processo riceve l'input dal terminale e la shell rimane in attesa che il comando termini.

Per avviare un comando in background:

```
cat &
```

Il carattere:

```
&
```

chiede alla shell di avviare il comando come job in background.

jobs

```
jobs
```

mostra i job gestiti dalla shell corrente.

Per esempio:

```
[1]+  Running      comando &
```

Il numero:

```
[1]
```

è il **job ID**, che è diverso dal PID.

jobs -l

```
jobs -l
```

mostra anche il PID dei processi associati ai job.

jobs -p

```
jobs -p
```

mostra i PID dei job.

jobs -r

```
jobs -r
```

mostra i job in esecuzione.

jobs -s

```
jobs -s
```

mostra i job sospesi.

fg — Foreground

```
fg
```

porta il job più recente in foreground.

Per portare un job specifico:

```
fg %2
```

Il simbolo `%` indica che `2` è un job ID, non un PID.

Ctrl + Z

La combinazione:

```
Ctrl + Z
```

invia normalmente:

```
SIGTSTP
```

al processo in foreground, sospendendolo.

Il job può poi essere visualizzato con:

```
jobs
```

bg — Background

Dopo aver sospeso un processo:

```
Ctrl + Z
```

è possibile riprenderlo in background:

```
bg
```

oppure:

```
bg %2
```

La shell utilizza quindi il meccanismo dei job per gestire il processo.

Segnali

Un **segnale** è una forma di comunicazione asincrona utilizzata dal sistema operativo e dai processi per notificare eventi o richiedere determinate azioni.

Esempi:

SIGTERM
SIGKILL
SIGINT
SIGSTOP
SIGCONT
SIGHUP

Un processo può:

- gestire alcuni segnali;
- ignorare alcuni segnali;
- utilizzare l'azione predefinita;
- non poter gestire determinati segnali.

Segnali principali

Segnale	Numero tipico Linux	Significato
SIGHUP	1	Hangup
SIGINT	2	Interruzione da terminale, normalmente <code>Ctrl+C</code>
SIGQUIT	3	Quit, normalmente con core dump
SIGILL	4	Istruzione illegale
SIGTRAP	5	Trap/debug
SIGABRT	6	Abort
SIGBUS	7	Errore di accesso al bus
SIGFPE	8	Errore aritmetico
SIGKILL	9	Termina immediatamente il processo
SIGSEGV	11	Accesso non valido alla memoria
SIGPIPE	13	Scrittura su pipe senza lettore
SIGALRM	14	Timer scaduto
SIGTERM	15	Richiesta di terminazione
SIGCHLD	17	Processo figlio terminato/cambiato stato
SIGCONT	18	Continua un processo sospeso
SIGSTOP	19	Sospende il processo
SIGTSTP	20	Sospensione da terminale, normalmente <code>Ctrl+Z</code>
SIGTTIN	21	Processo in background tenta di leggere dal terminale
SIGTTOU	22	Processo in background tenta di scrivere sul terminale in determinate condizioni

I numeri possono variare tra sistemi operativi e architetture. Quelli indicati sono quelli comunemente utilizzati su Linux.

I segnali più importanti da ricordare sono:

```
SIGINT → 2
SIGKILL → 9
SIGTERM → 15
SIGSTOP → 19
SIGCONT → 18
```

SIGTERM VS **SIGKILL**

Questa distinzione è molto importante.

SIGTERM

```
SIGTERM = 15
```

chiede al processo di terminare.

Il processo può gestire il segnale e svolgere operazioni di pulizia prima di terminare.

È generalmente preferibile utilizzare **SIGTERM** prima di ricorrere a **SIGKILL**.

SIGKILL

```
SIGKILL = 9
```

termina immediatamente il processo.

Il processo non può intercettare, ignorare o gestire **SIGKILL**.

Per questo motivo dovrebbe essere utilizzato con cautela.

SIGSTOP

```
SIGSTOP = 19
```

sospende il processo.

Come **SIGKILL**, non può essere intercettato o ignorato dal processo.

SIGCONT

```
SIGCONT = 18
```

richiede la continuazione di un processo precedentemente sospeso.

SIGINT

```
SIGINT = 2
```

viene normalmente generato quando l'utente preme:

```
Ctrl + C
```

ed è una richiesta di interrompere il processo in foreground.

kill

Il comando:

```
kill
```

viene utilizzato per inviare segnali ai processi.

Non significa necessariamente "uccidere": il comportamento dipende dal segnale specificato.

Per esempio:

```
kill -TERM 1234
```

invia **SIGTERM** al processo con PID **1234**.

kill -l

Per visualizzare i segnali disponibili:

```
kill -l
```

L'opzione corretta è **-l** con la lettera L minuscola, non **-I**.

Convertire nome e numero del segnale

```
kill -l TERM
```

può restituire:

```
15
```

Mentre:

```
kill -l 15
```

restituisce il relativo nome del segnale.

Specificare il segnale

```
kill -s SIGTERM 1234
```

invia `SIGTERM` al PID `1234`.

È possibile utilizzare anche:

```
kill -15 1234
```

oppure:

```
kill -TERM 1234
```

Permessi per inviare segnali

Normalmente un utente può inviare segnali ai processi che possiede.

Per inviare segnali a processi appartenenti ad altri utenti possono essere necessari privilegi aggiuntivi, normalmente `root` o un'apposita autorizzazione.

`killall`

È possibile utilizzare:

```
killall nome_processo
```

per inviare un segnale ai processi corrispondenti al nome specificato.

Per esempio:

```
killall firefox
```

`killall` può interessare più processi contemporaneamente, quindi va utilizzato con attenzione.

File descriptor

Ogni processo dispone normalmente di tre file descriptor standard:

```
0 → stdin  
1 → stdout  
2 → stderr
```

Standard input — `stdin`

```
0
```

rappresenta l'input standard.

Normalmente è collegato alla tastiera del terminale.

Standard output — `stdout`

```
1
```

rappresenta l'output standard.

Normalmente viene visualizzato sul terminale.

Standard error — `stderr`

```
2
```

rappresenta l'output degli errori.

Normalmente viene anch'esso visualizzato sul terminale, ma è un flusso distinto da `stdout`.

Redirezione di `stdin`

```
comando < file.txt
```

lo standard input del comando viene preso dal file invece che dalla tastiera.

Esempio:

```
sort < nomi.txt
```

Redirezione di `stdout`

```
comando > file.txt
```

invia lo standard output al file.
Se il file non esiste, viene creato.
Se esiste, viene sovrascritto.

Append di `stdout`

```
comando >> file.txt
```

aggiunge l'output alla fine del file.

Redirezione di `stderr`

```
comando 2> errori.txt
```

redireziona solamente lo standard error.

Redirezione di `stdout` e `stderr`

```
comando > output.txt 2>&1
```

significa:

1. manda `stdout` nel file;
2. manda `stderr` nello stesso posto a cui è stato collegato `stdout`.

Una forma Bash equivalente e più compatta è:

```
comando &> output.txt
```

Pipe tra processi

```
comando1 | comando2
```

collega:

```
stdout di comando1
    ↓
stdin di comando2
```

Esempio:

```
cat /etc/passwd | grep root
```

|&

In Bash:

```
comando1 |& comando2
```

collega sia:

```
stdout
```

sia:

```
stderr
```

del primo comando allo standard input del secondo.

È equivalente, concettualmente, a:

```
comando1 2>&1 | comando2
```

exec e redirectione persistente

Il comando:

```
exec
```

può essere utilizzato per modificare i file descriptor della shell corrente.

Per esempio:

```
exec > ~/output.txt
```

da quel momento, lo standard output della shell viene rediretto verso `output.txt`.

Per ripristinare l'output verso il terminale è necessario conoscere il file descriptor originale e salvarlo prima della redirectione.

Un esempio più sicuro è:

```
exec 3>&1
```

```
exec > ~/output.txt
```

In questo caso `3` contiene una copia del vecchio `stdout`.

Per ripristinarlo:

```
exec 1>&3
```

e successivamente:

```
exec 3>&-
```

chiude il file descriptor `3`.

Duplicazione dei file descriptor

La sintassi:

```
n>&m
```

duplica il file descriptor `m` nel file descriptor `n`.

Esempio:

```
2>&1
```

significa:

```
stderr (2) → stesso destinatario di stdout (1)
```

È una tecnica molto importante per comprendere le redirezioni Bash.

Processi in foreground e background

Per avviare:

```
comando &
```

la shell restituisce normalmente il controllo all'utente mentre il processo continua in background.

Per esempio:

```
sleep 100 &
```

La shell può mostrare qualcosa come:

```
[1] 1234
```

dove:

```
[1] → job ID  
1234 → PID
```
